

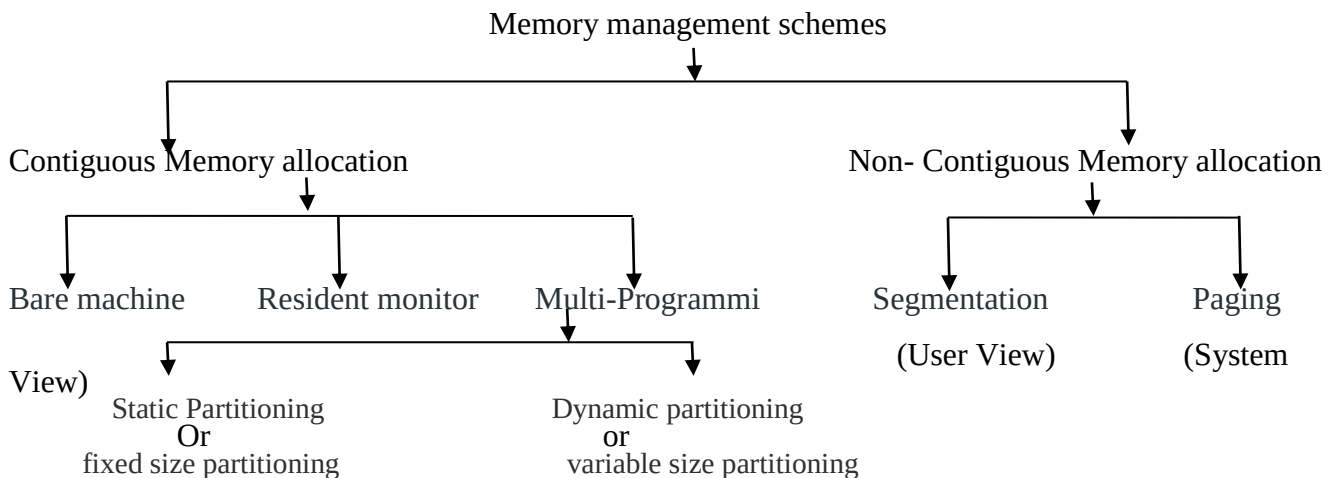
UNIT-4 (MEMORY MANAGEMENT)

Memory management is an operating system function that controls and coordinates computer memory (RAM), allocating, tracking, and freeing memory blocks for running processes to optimize performance. It manages the movement of processes between main memory and disk, ensuring efficient allocation, addressing fragmentation, and supporting virtual memory techniques.

Key aspects and techniques of memory management include:

- **Memory Tracking:** Monitoring every location, whether it is allocated or free.
- **Allocation/Deallocation:** Allocating memory to processes when they start and freeing it when they terminate.
- **Contiguous Allocation:** Placing a process in a single, unbroken block of memory.
- **Non-contiguous Allocation (Paging/Segmentation):** Breaking programs into pieces (pages or segments) and loading them into non-contiguous physical memory to reduce fragmentation.
- **Virtual Memory:** A technique that allows the execution of processes that are not entirely in main memory, often by swapping processes between memory and disk.
- **Memory Management Unit (MMU):** A hardware component that translates logical (virtual) addresses generated by the CPU into physical addresses in RAM.

Memory management schemes are broadly divided into the following categories:



Bare Machine and Resident Monitor

Bare Machine:

Bare machine is logical hardware which is used to execute the program in the processor without using the operating system. we have studied that we can't execute any process without the Operating system. But yes with the help of the Bare machine we can do that.

Initially, when the operating systems are not developed, the execution of an instruction is done by directly on hardware without using any interfering hardware, at that time the only drawback was that the Bare machines accepting the instruction in only machine language, due to this those person who has sufficient knowledge about Computer field are able to operate a computer. so after the development of the operating system Bare machine is referred to as inefficient.

Resident Monitor:

if we talk about how the code runs on Bare machines, then this component is used, so basically, the Resident

Monitor is a code that runs on Bare Machines. The resident monitor works like an operating system that controls the instructions and performs all necessary functions. It also works like job sequencer because it also sequences the job and sends them to the processor. After scheduling the job Resident monitors loads the programs one by one into the main memory according to their sequences. One most important factor about the resident monitor is that when the program execution occurred there is no gap between the program execution and the processing is going to be faster. The Resident monitors are divided into 4 parts as:

1. Control Language Interpreter
2. Loader
3. Device Driver
4. Interrupt Processing

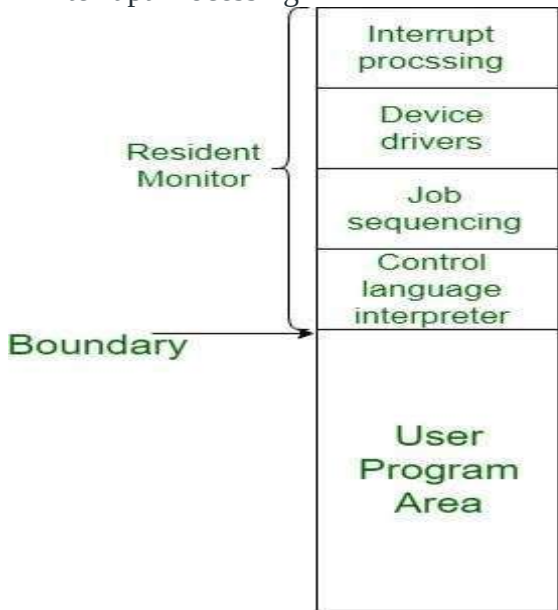


Fig.: Memory Layout of Resident Monitor

These are explained as following below.

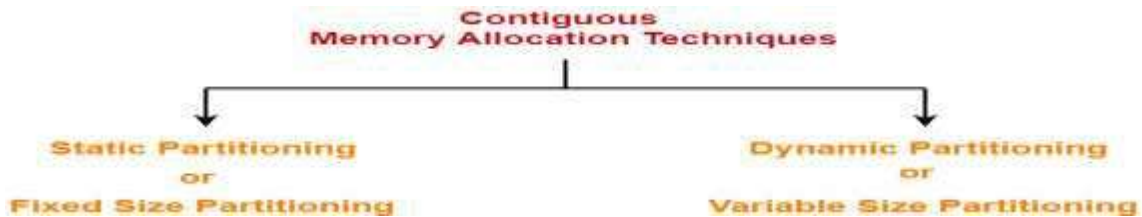
1. **Control Language Interpreter:**
The first part of the Resident monitor is control language interpreter which is used to read and carry out the instruction from one level to the next level.
2. **Loader:**
The second part of the Resident monitor which is the main part of the Resident Monitor is Loader which Loads all the necessary system and application programs into the main memory.
3. **Device Driver:**
The third part of the Resident monitor is Device Driver which is used to manage the connecting input-output devices to the system. So basically it is the interface between the user and the system. it works as an interface between the request and response. request which user made, Device driver responds that the system produces to fulfill these requests.
4. **Interrupt Processing:**
The fourth part as the name suggests, it processes the all occurred interrupt to the system.

Contiguous Memory Allocation-

- Contiguous memory allocation is a memory allocation technique.
- It allows to store the process only in a contiguous fashion.
- Thus, entire process has to be stored as a single entity at one place inside the memory.

Techniques-

There are two popular techniques used for contiguous memory allocation-



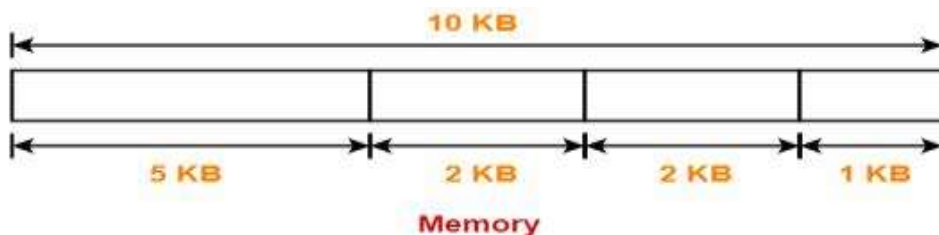
1. Static Partitioning
2. Dynamic Partitioning

Static Partitioning-

- Static partitioning is a fixed size partitioning scheme.
- In this technique, main memory is pre-divided into fixed size partitions.
- The size of each partition is fixed and can not be changed.
- Each partition is allowed to store only one process.

Example-

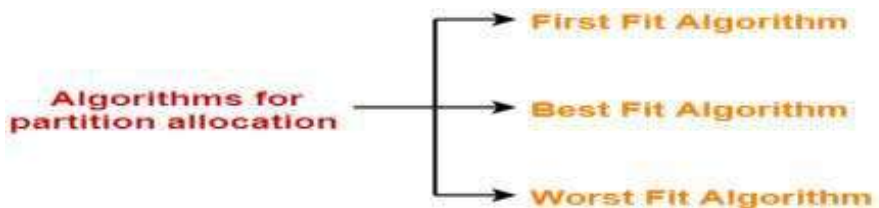
Under fixed size partitioning scheme, a memory of size 10 KB may be divided into fixed size partitions as-



- These partitions are allocated to the processes as they arrive.
- The partition allocated to the arrived process depends on the algorithm followed.

Algorithms for Partition Allocation-

Popular algorithms used for allocating the partitions to the arriving processes are-



1. First Fit Algorithm
2. Best Fit Algorithm
3. Worst Fit Algorithm

1. First Fit Algorithm-

- This algorithm starts scanning the partitions serially from the starting.
- When an empty partition that is big enough to store the process is found, it is allocated to the process.
- Obviously, the partition size has to be greater than or at least equal to the process size.

2. Best Fit Algorithm-

- This algorithm first scans all the empty partitions.
- It then allocates the smallest size partition to the process.

3. Worst Fit Algorithm-

- This algorithm first scans all the empty partitions.
- It then allocates the largest size partition to the process.

Important Points-

Point-01:

For static partitioning,

- Best Fit Algorithm works best.
- This is because space left after the allocation inside the partition is of very small size.
- Thus, internal fragmentation is least.

Point-02:

For static partitioning,

- Worst Fit Algorithm works worst.
- This is because space left after the allocation inside the partition is of very large size.
- Thus, internal fragmentation is maximum.

Internal Fragmentation

- It occurs when the space is left inside the partition after allocating the partition to a process.
- This space is called as internally fragmented space.
- This space can not be allocated to any other process.
- This is because only static partitioning allows to store only one process in each partition.
- Internal Fragmentation occurs only in static partitioning.

External Fragmentation

- It occurs when the total amount of empty space required to store the process is available in the main memory.
- But because the space is not contiguous, so the process can not be stored.

Translating Logical Address into Physical Address-

- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Following steps are followed to translate logical address into physical address-

Step-01:

- The translation scheme uses two registers that are under the control of operating system.
- During context switching, the values corresponding to the process being loaded are set in the registers.

These two registers are-

- Relocation Register
- Limit Register



- Relocation Register stores the base address or starting address of the process in the main memory.
- Limit Register stores the size or length of the process.

Step-02:

- CPU generates a logical address containing the address of the instruction that it wants to read.

Step-03:

- The logical address generated by the CPU is compared with the limit of the process.
- Now, two cases are possible-

Case-01: Generated Address $\geq$ Limit

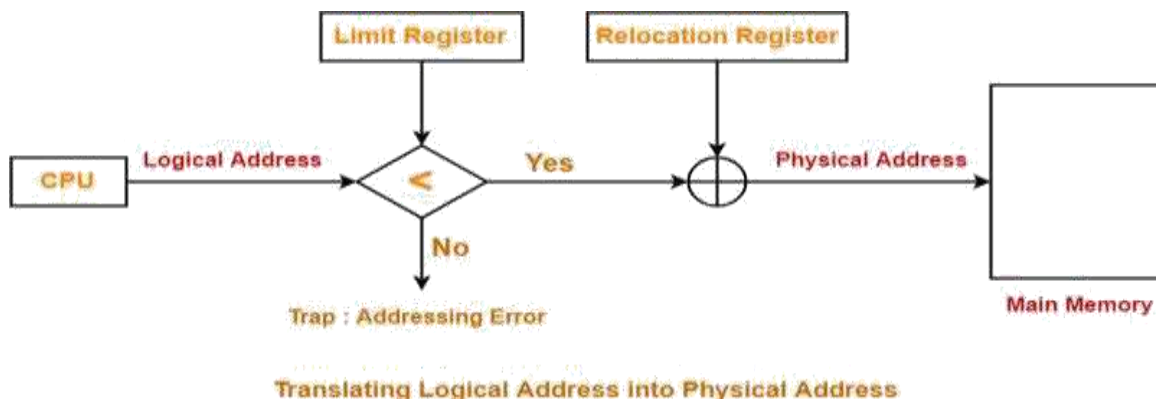
- If address is found to be greater than or equal to the limit, a trap is generated.
- This helps to prevent unauthorized access.

Case-02: Generated Address $<$ Limit

- The address must always lie in the range $[0, \text{limit}-1]$.
- If address is found to be smaller than the limit, then the request is treated as a valid request.
- Then, generated address is added with the base address of the process.
- The result obtained after addition is the address of the memory location storing the required word.

Diagram-

The following diagram illustrates the above steps of translating logical address into physical address-



Advantages-

The advantages of static partitioning are-

- It is simple and easy to implement.
- It supports multiprogramming since multiple processes can be stored inside the main memory.
- Only one memory access is required which reduces the access time.

Disadvantages-

The disadvantages of static partitioning are-

- It suffers from both internal fragmentation and external fragmentation.
- It utilizes memory inefficiently.
- The degree of multiprogramming is limited equal to number of partitions.
- There is a limitation on the size of process since processes with size greater than the size of largest partition can't be stored and executed.

Dynamic Partitioning-

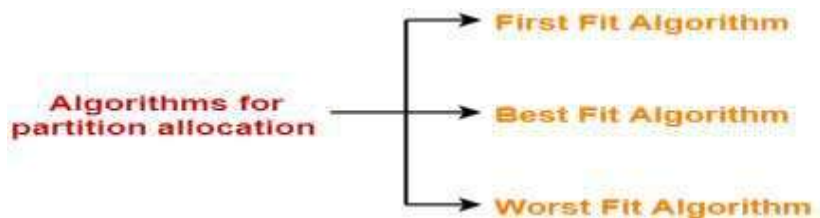
- Dynamic partitioning is a variable size partitioning scheme.
- It performs the allocation dynamically.
- When a process arrives, a partition of size equal to the size of process is created.
- Then, that partition is allocated to the process.

Partition Allocation Algorithms-

- The processes arrive and leave the main memory.
- As a result, holes of different size are created in the main memory.
- These holes are allocated to the processes that arrive in future.

Partition allocation algorithms are used to decide which hole should be allocated to the arrived process.

Popular partition allocation algorithms are-



1. First Fit Algorithm
2. Best Fit Algorithm
3. Worst Fit Algorithm

Important Points-

Point-01:

For dynamic partitioning,

- Worst Fit Algorithm works best.
- This is because space left after allocation inside the partition is of large size.
- There is a high probability that this space might suit the requirement of arriving processes.

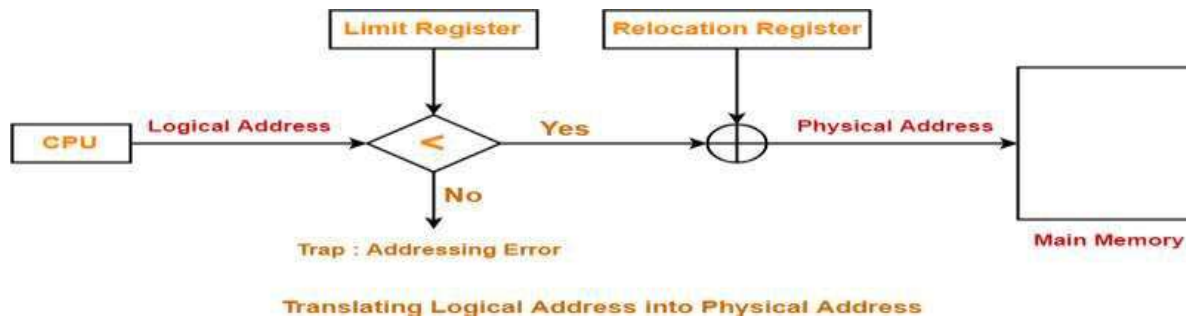
Point-02:

For dynamic partitioning,

- Best Fit Algorithm works worst.
- This is because space left after allocation inside the partition is of very small size.
- There is a low probability that this space might suit the requirement of arriving processes.

Translating Logical Address into Physical Address-

The following diagram illustrates the steps of translating logical address into physical address-



Advantages-

The advantages of dynamic partitioning are-

- It does not suffer from internal fragmentation.
- Degree of multiprogramming is dynamic.
- There is no limitation on the size of processes.

Disadvantages-

The disadvantages of dynamic partitioning are-

- It suffers from external fragmentation.
- Allocation and deallocation of memory is complex.

PRACTICE PROBLEMS BASED ON CONTIGUOUS MEMORY ALLOCATION-

Problem-01:

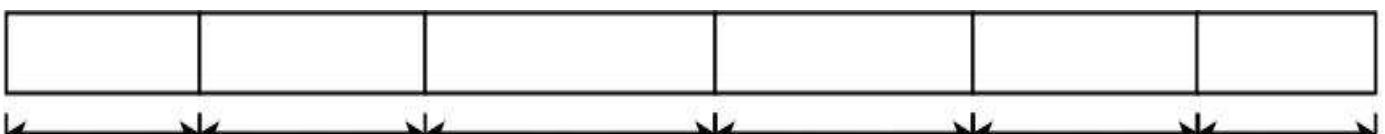
Consider six memory partitions of size 200 KB, 400 KB, 600 KB, 500 KB, 300 KB and 250 KB. These partitions need to be allocated to four processes of sizes 357 KB, 210 KB, 468 KB and 491 KB in that order. Perform the allocation of processes using-

1. First Fit Algorithm
2. Best Fit Algorithm
3. Worst Fit Algorithm

Solution-

According to question,

The main memory has been divided into fixed size partitions as-



Let us say the given processes are-

- Process P1 = 357 KB
- Process P2 = 210 KB
- Process P3 = 468 KB
- Process P4 = 491 KB

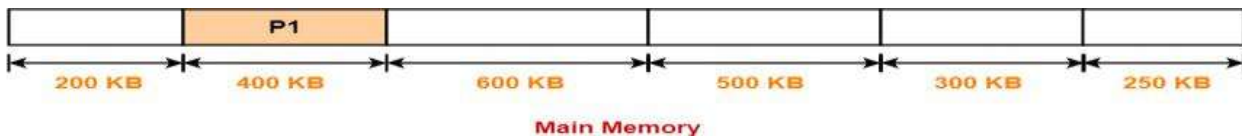
Allocation Using First Fit Algorithm-

In First Fit Algorithm,

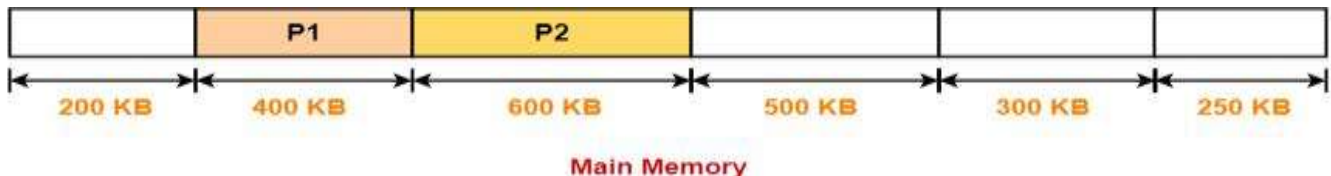
- Algorithm starts scanning the partitions serially.
- When a partition big enough to store the process is found, it allocates that partition to the process.

The allocation of partitions to the given processes is shown below-

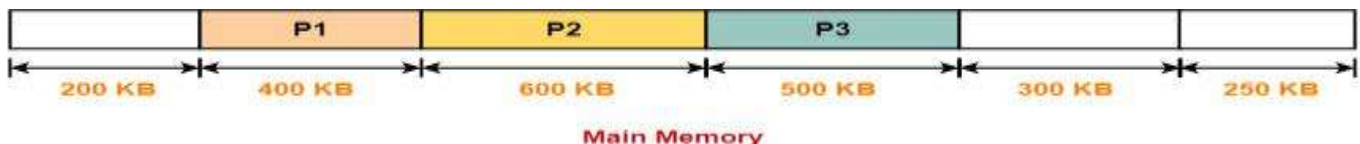
Step-01:



Step-02:



Step-03:



Step-04:

- Process P4 can not be allocated the memory.
- This is because no partition of size greater than or equal to the size of process P4 is available.

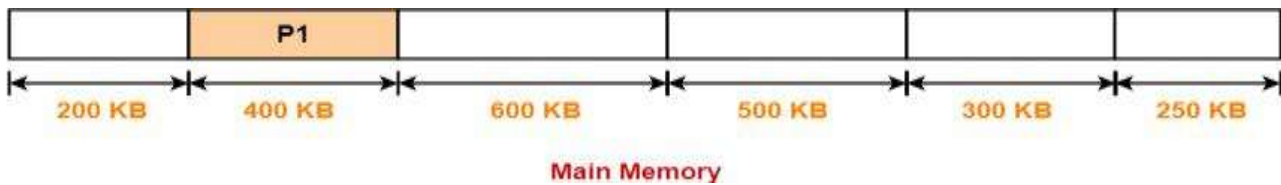
Allocation Using Best Fit Algorithm-

In Best Fit Algorithm,

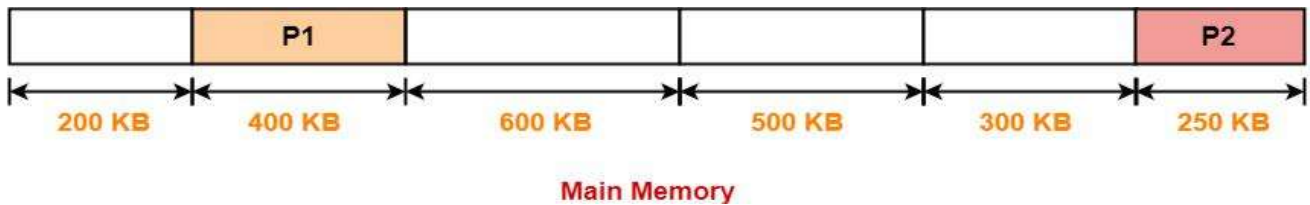
- Algorithm first scans all the partitions.
- It then allocates the partition of smallest size that can store the process.

The allocation of partitions to the given processes is shown below-

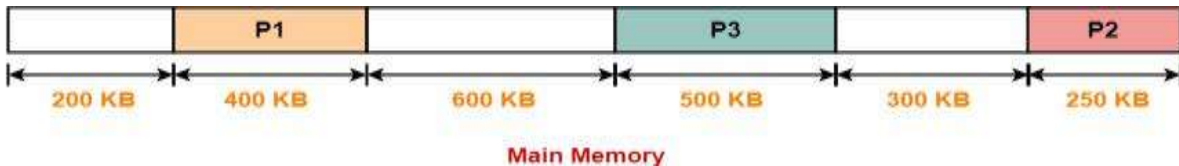
Step-01:



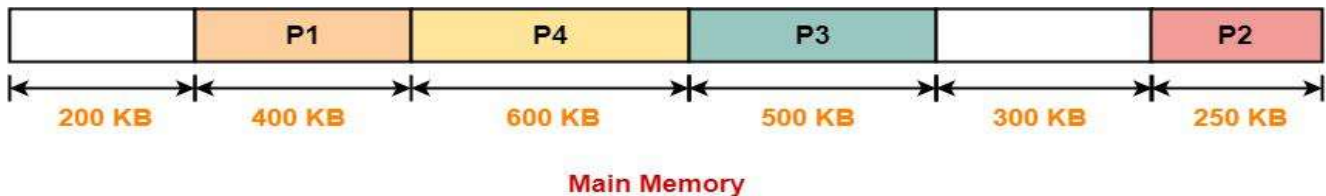
Step-02:



Step-03:



Step-04:



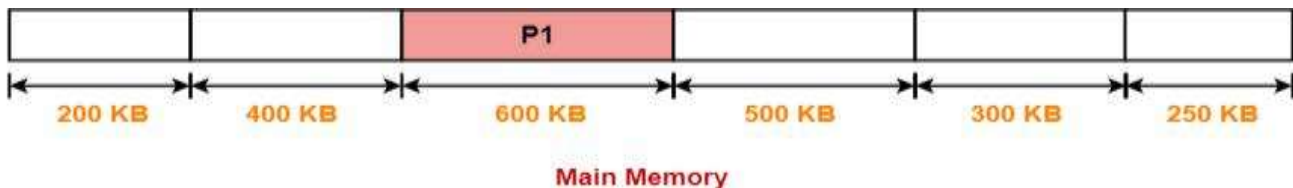
Allocation Using Worst Fit Algorithm-

In Worst Fit Algorithm,

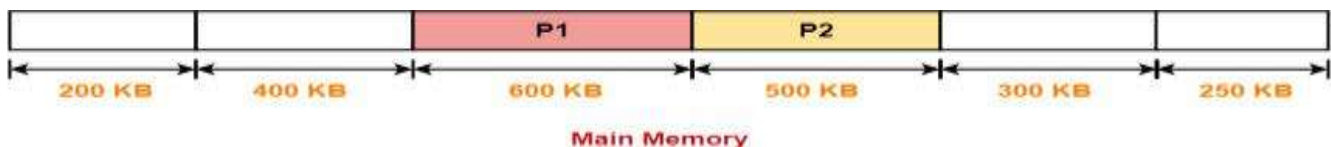
- Algorithm first scans all the partitions.
- It then allocates the partition of largest size to the process.

The allocation of partitions to the given processes is shown below-

Step-01:



Step-02:



Step-03:

- Process P3 and Process P4 can not be allocated the memory.
- This is because no partition of size greater than or equal to the size of process P3 and process P4 is available.

Difference between Fixed Partitioning and Variable Partitioning :

S.NO.	Fixed partitioning	Variable partitioning
1.	In multi-programming with fixed partitioning the main memory is divided into fixed sized partitions.	In multi-programming with variable partitioning the main memory is not divided into fixed sized partitions.
2.	Only one process can be placed in a partition.	In variable partitioning, the process is allocated a chunk of free memory.
3.	It does not utilize the main memory effectively.	It utilizes the main memory effectively.
4.	There is presence of internal fragmentation and external fragmentation.	There is external fragmentation.
5.	Degree of multi-programming is less.	Degree of multi-programming is higher.
6.	It is more easier to implement.	It is less easier to implement.
7.	There is limitation on size of process.	There is no limitation on size of process.

Logical and Physical Address Space in Operating Systems

The address that is generated by the CPU is commonly referred to as the **Logical Address**. It is basically a virtual address. The logical address is basically the address of an instruction or data used by any program. The set of all logical addresses that are generated by any program is referred to as **Logical Address Space**.

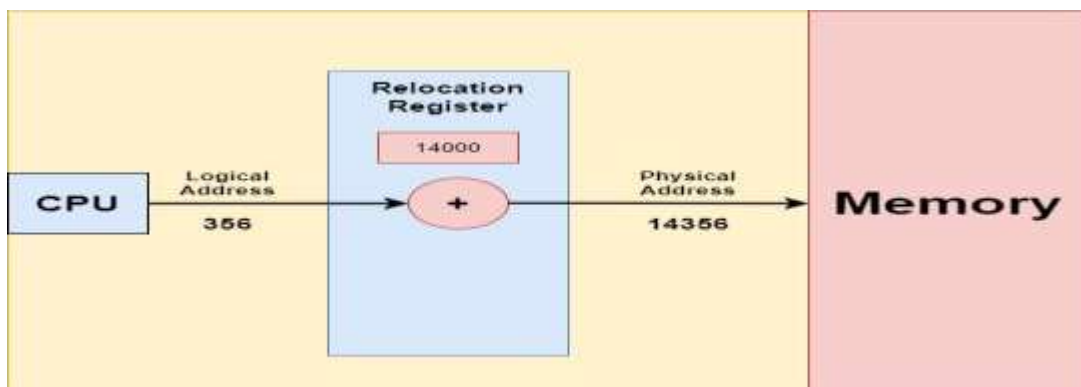
The address that is loaded into the **memory-address register** of the memory is commonly referred to as a **Physical address**. A physical address cannot be accessed by the user directly but the user can calculate the physical address with the help of a **Logical address**.

The user's program mainly generates the logical address and the user thinks that the program is running in this logical address but the program mainly needs physical memory in order to complete its execution.

The set of all **physical addresses** corresponding to the **Logical addresses** is commonly known as **Physical Address Space**.

Memory Management Unit(MMU) in OS

It is a hardware device that does the run-time mapping from the virtual address to the physical address. It is located within the Central Processing Unit.



In the above diagram, the base register is termed the **Relocation register**. The relocation register is a special register in the CPU and is used for the mapping of logical addresses used by a program to physical addresses of the system's main memory.

The value in the relocation register is added to every address that is generated by the user process at the time when the address is sent to the memory.

MMU Example

Suppose the **base is at 14000**, then an attempt by the user to address location **0** is relocated dynamically to **14000**; thus access to location **356** is mapped to **14356**.

It is important to note that the user program never sees the real physical addresses. The Program can create a pointer to location 356 and store it in the memory and then manipulate it after that compare it with other addresses as number 356.

User program always deals with the logical addresses. The **Memory Mapping unit** mainly converts the logical addresses into the physical addresses. The final location of the referenced memory address is not determined **until the reference is made**.

Difference Between Logical Address and Physical Address

Let us now cover the differences between the Logical addresses and Physical addresses in the Operating System

S.No	Logical Address	Physical Address
1.	Users can access the logical address of the Program.	User can never access the physical address of the Program
2.	The logical address is generated by the CPU.	The physical address is located in the memory unit.
3.	The user can access the physical address with the help of a logical address.	A physical address can be accessed by a user indirectly but not directly.
4.	The logical address does not exist physically	On the other hand, the physical address is a
	in the memory and thus termed as a Virtual address.	location in the memory. Thus it can be accessed physically.
5.	The set of all logical addresses that are generated by any program is referred to as Logical Address Space .	The set of all physical addresses corresponding to the Logical addresses is commonly known as Physical Address Space .
6.	This address is generated by the CPU.	It is computed by the Memory Management Unit(MMU).

Memory Protection scheme

Memory protection is a strategy that makes it possible to manage the amount of access rights that are granted to the memory found on a computer **hard drive**. The main purpose of this type of protection is to minimize the potential for some type of storage violation that would harm the data contained in the memory, or damage a portion of the memory capacity of the hard drive.

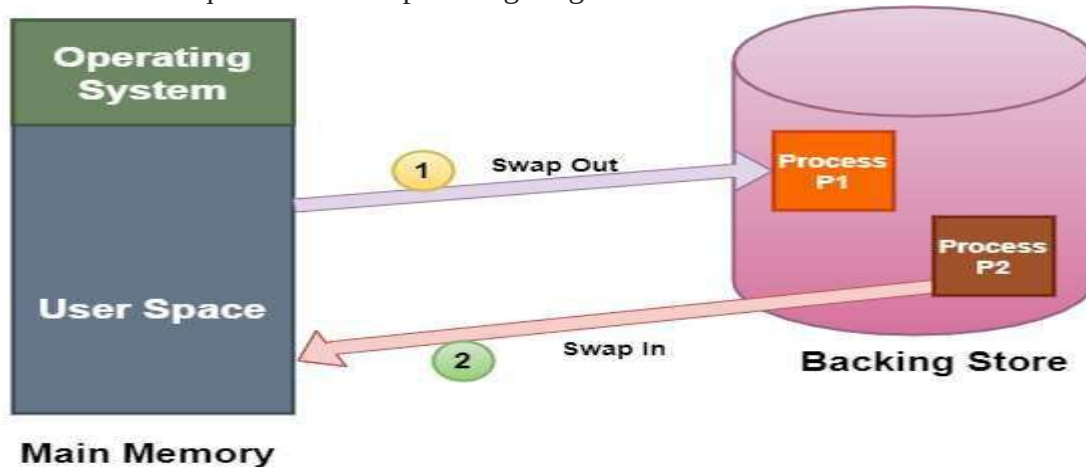
One of the main functions of memory protection is the prevention of any application from making use of memory that the **operating system** has not specifically allocated to that application. This prevents applications from seizing control of an inordinate amount of memory and possibly causing damage that negatively impacts other applications that are currently in use, or even creating a loss of data that is saved on the hard drive. In many operating systems, this is managed by segmenting the memory for use by all open applications, ensuring that each has enough to operate properly without creating issues with the other running applications.

Swapping

Swapping is a memory management technique and is used to temporarily remove the inactive programs from the main memory of the computer system. Any process must be in the memory for its execution, but can be swapped temporarily out of memory to a backing store and then again brought back into the memory to complete its execution. Swapping is done so that other processes get memory for their execution.

Due to the swapping technique performance usually gets affected, but it also helps in running multiple and big processes in parallel. **The swapping** process is also known as a technique for **memory compaction**. Basically, low priority processes may be swapped out so that processes with a higher priority may be loaded and executed.

Let us understand this technique with the help of a figure given below:



The above diagram shows swapping of two processes where the disk is used as a Backing store.

In the above diagram, suppose there is a multiprogramming environment with a round-robin scheduling algorithm; whenever the time quantum expires then the memory manager starts to swap out those processes that are just finished and swap another process into the memory that has been freed. And in the meantime, the CPU scheduler allocates the time slice to some other processes in the memory.

The swapping of processes by the memory manager is fast enough that some processes will be in memory, ready to execute, when the CPU scheduler wants to reschedule the CPU.

A variant of the swapping technique is the priority-based scheduling algorithm. If any higher-priority process arrives and wants service, then the memory manager swaps out lower priority processes and then load the higher priority processes and then execute them. When the process with higher priority finishes .then the process with lower priority swapped back in and continues its execution. This variant is sometimes known as roll in and roll out.

There are two more concepts that come in the swapping technique and these are: **swap in** and **swap out**.

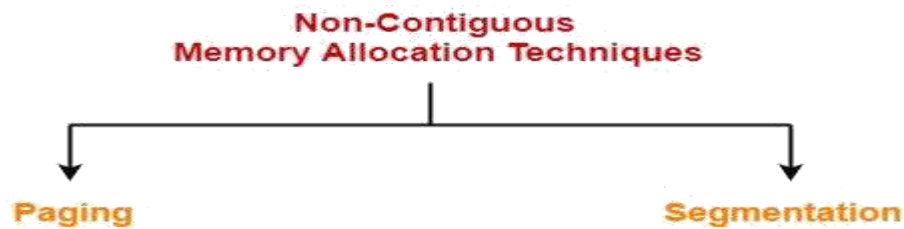
Swap In and Swap Out

The procedure by which any process gets removed from the **hard disk** and placed in the **main memory or RAM** commonly known as **Swap In**.

On the other hand, **Swap Out** is the method of removing a process from the **main memory or RAM** and then adding it to the **Hard Disk**.

Advantages of Swapping

The advantages/benefits of the Swapping technique are as follows:



1. The swapping technique mainly helps the CPU to manage multiple processes within a single main memory.
2. This technique helps to create and use virtual memory.
3. With the help of this technique, the CPU can perform several tasks simultaneously. Thus, processes need not wait too long before their execution.
4. This technique is economical.
5. This technique can be easily applied to priority-based scheduling in order to improve its performance.

Disadvantages of Swapping

The drawbacks of the swapping technique are as follows:

1. There may occur inefficiency in the case if a resource or a variable is commonly used by those processes that are participating in the swapping process.
2. If the algorithm used for swapping is not good then the overall method can increase the number of page faults and thus decline the overall performance of processing.
3. If the computer system loses power at the time of high swapping activity then the user might lose all the information related to the program.

Non-Contiguous Memory Allocation-

- Non-contiguous memory allocation is a memory allocation technique.
- It allows to store parts of a single process in a non-contiguous fashion.
- Thus, different parts of the same process can be stored at different places in the main memory.

Techniques-

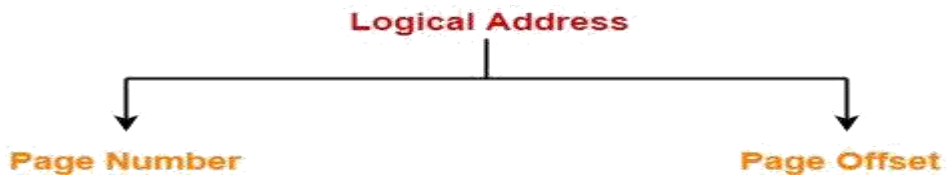
There are two popular techniques used for non-contiguous memory allocation-

1. Paging
2. Segmentation

Paging-

- Paging is a fixed size partitioning scheme.
- In paging, secondary memory and main memory are divided into equal fixed size partitions.
- The partitions of secondary memory are called as **pages**.
- The partitions of main memory are called as **frames**.





- Each process is divided into parts where size of each part is same as page size.
- The size of the last part may be less than the page size.
- The pages of process are stored in the frames of main memory depending upon their availability.

Example-

- Consider a process is divided into 4 pages P₀, P₁, P₂ and P₃.
- Depending upon the availability, these pages may be stored in the main memory frames in a non-contiguous fashion as shown-

P1
P3
P0
P2

Main Memory

Translating Logical Address into Physical Address-

- CPU always generates a logical address.
 - A physical address is needed to access the main memory.
- Following steps are followed to translate logical address into physical address-

Step-01: CPU generates a logical address consisting of two parts-

1. Page Number
 2. Page Offset
- Page Number specifies the specific page of the process from which CPU wants to read the data.
 - Page Offset specifies the specific word on the page that CPU wants to read.

Step-02: For the page number generated by the CPU,

- **Page Table** provides the corresponding frame number (base address of the frame) where that page is stored in the main memory.

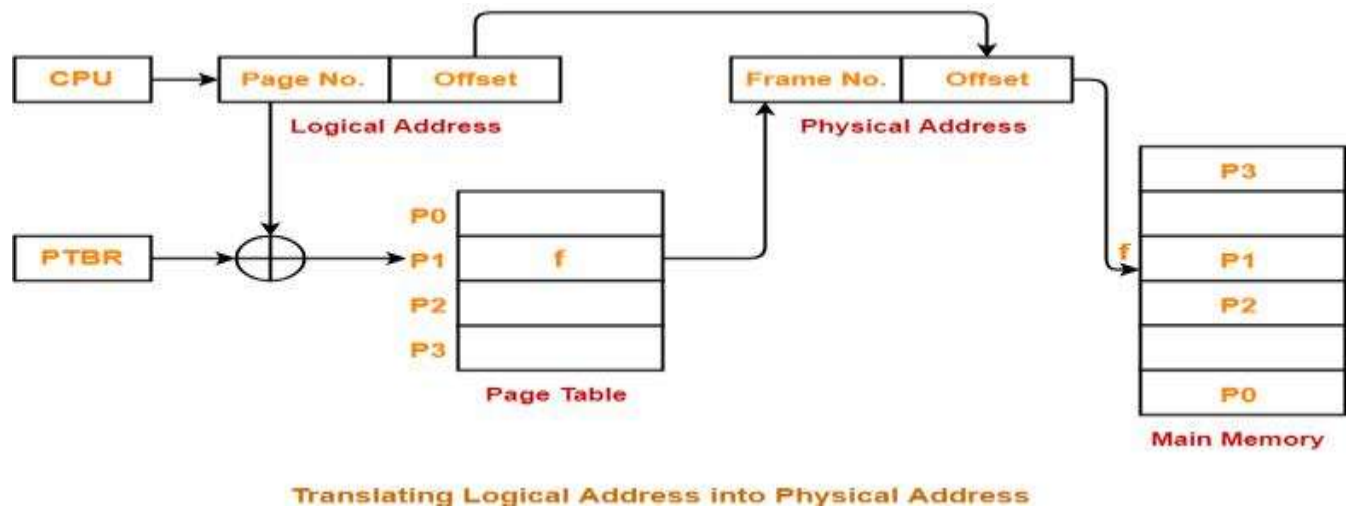
Step-03: The frame number combined with the page offset forms the required physical address.



- Frame number specifies the specific frame where the required page is stored.
- Page Offset specifies the specific word that has to be read from that page.

Diagram-

The following diagram illustrates the above steps of translating logical address into physical address-



PTBR(page table base register)

Advantages-

The advantages of paging are-

- It allows to store parts of a single process in a non-contiguous fashion.
- It solves the problem of external fragmentation.

Disadvantages-

The disadvantages of paging are-

- It suffers from internal fragmentation.
- There is an overhead of maintaining a page table for each process.
- The time taken to fetch the instruction increases since now two memory accesses are required.

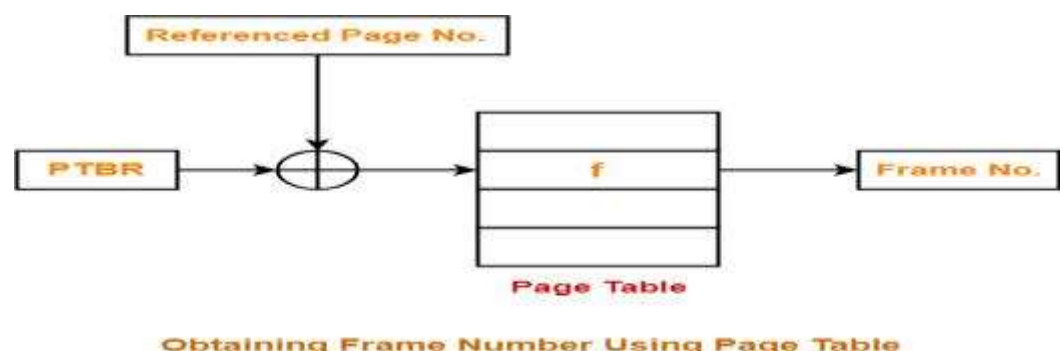
Page Table-

- Page table is a data structure.
- It maps the page number referenced by the CPU to the frame number where that page is stored.

Characteristics-

- Page table is stored in the main memory.
- Number of entries in a page table = Number of pages in which the process is divided.
- Page Table Base Register (PTBR) contains the base address of page table.
- Each process has its own independent page table.

Working-



- Page Table Base Register (PTBR) provides the base address of the page table.
- The base address of the page table is added with the page number referenced by the CPU.
- It gives the entry of the page table containing the frame number where the referenced page is stored.

Page Table Entry-

- A page table entry contains several information about the page.
- The information contained in the page table entry varies from operating system to operating system.
- The most important information in a page table entry is frame number.

In general, each entry of a page table contains the following information-



Page Table Entry Format

1. Frame Number-

- Frame number specifies the frame where the page is stored in the main memory.
- The number of bits in frame number depends on the number of frames in the main memory.

2. Present / Absent Bit-

- This bit is also sometimes called as **valid / invalid bit**.
- This bit specifies whether that page is present in the main memory or not.
- If the page is not present in the main memory, then this bit is set to 0 otherwise set to 1.

NOTE

- If the required page is not present in the main memory, then it is called as **Page Fault**.
- A page fault requires page initialization.
- The required page has to be initialized (fetched) from the secondary memory and brought into the main memory.

3. Protection Bit-

- This bit is also sometimes called as "**Read / Write bit**".
- This bit is concerned with the page protection.
- It specifies the permission to perform read and write operation on the page.
- If only read operation is allowed to be performed and no writing is allowed, then this bit is set to 0.
- If both read and write operation are allowed to be performed, then this bit is set to 1.

4. Reference Bit-

- Reference bit specifies whether that page has been referenced in the last clock cycle or not.
- If the page has been referenced recently, then this bit is set to 1 otherwise set to 0.

NOTE

- Reference bit is useful for page replacement policy.
- A page that has not been referenced recently is considered a good candidate for page replacement in LRU page replacement policy.

5. Caching Enabled / Disabled-

- This bit enables or disables the caching of page.
- Whenever freshness in the data is required, then caching is disabled using this bit.
- If caching of the page is disabled, then this bit is set to 1 otherwise set to 0.

6. Dirty Bit-

- This bit is also sometimes called as “**Modified bit**“. This bit specifies whether that page has been modified or not.
- If the page has been modified, then this bit is set to 1 otherwise set to 0.

NOTE

In case the page is modified,

- Before replacing the modified page with some other page, it has to be written back in the secondary memory to avoid losing the data.
- Dirty bit helps to avoid unnecessary writes.
- This is because if the page is not modified, then it can be directly replaced by another page without any need of writing it back to the disk.

Important Formulas-

The following list of formulas is very useful for solving the numerical problems based on paging.

For Main Memory-

- Physical Address Space = Size of main memory
- Size of main memory = Total number of frames x Page size
- Frame size = Page size
- If number of frames in main memory = 2^X , then number of bits in frame number = X bits
- If Page size = 2^X Bytes, then number of bits in page offset = X bits
- If size of main memory = 2^X Bytes, then number of bits in physical address = X bits

For Process-

- Virtual Address Space = Size of process
- Number of pages the process is divided = Process size / Page size
- If process size = 2^X bytes, then number of bits in virtual address space = X bits

For Page Table-

- Size of page table = Number of entries in page table x Page table entry size
- Number of entries in pages table = Number of pages the process is divided
- Page table entry size = Number of bits in frame number + Number of bits used for optional fields if any

NOTE-

- In general, if the given address consists of 'n' bits, then using 'n' bits, 2^n locations are possible.
- Then, size of memory = $2^n \times$ Size of one location.
- If the memory is byte-addressable, then size of one location = 1 byte.
- Thus, size of memory = 2^n bytes.
- If the memory is word-addressable where 1 word = m bytes, then size of one location = m bytes.
- Thus, size of memory = $2^n \times m$ bytes.

PRACTICE PROBLEMS BASED ON PAGING AND PAGE TABLE-

Problem-01:

Calculate the size of memory if its address consists of 22 bits and the memory is 2-byte addressable.

Solution-

We have-

- Number of locations possible with 22 bits = 2^{22} locations
- It is given that the size of one location = 2 bytes

$$\begin{aligned}\text{Thus, Size of memory} &= 2^{22} \times 2 \text{ bytes} \\ &= 2^{23} \text{ bytes} \\ &= 8 \text{ MB}\end{aligned}$$

Problem-02:

Calculate the number of bits required in the address for memory having size of 16 GB. Assume the memory is 4-byte addressable.

Solution-

Let 'n' number of bits are required. Then, Size of memory = $2^n \times 4$ bytes.

Since, the given memory has size of 16 GB, so we have-

$$2^n \times 4 \text{ bytes} = 16 \text{ GB}$$

$$2^n \times 4 = 16 \text{ G}$$

$$2^n \times 2^2 = 2^{34}$$

$$2^n = 2^{32}$$

n = 32 bits

Segmentation-

- Like Paging, Segmentation is another non-contiguous memory allocation technique.
- In segmentation, process is not divided blindly into fixed size pages.
- Rather, the process is divided into modules for better visualization.

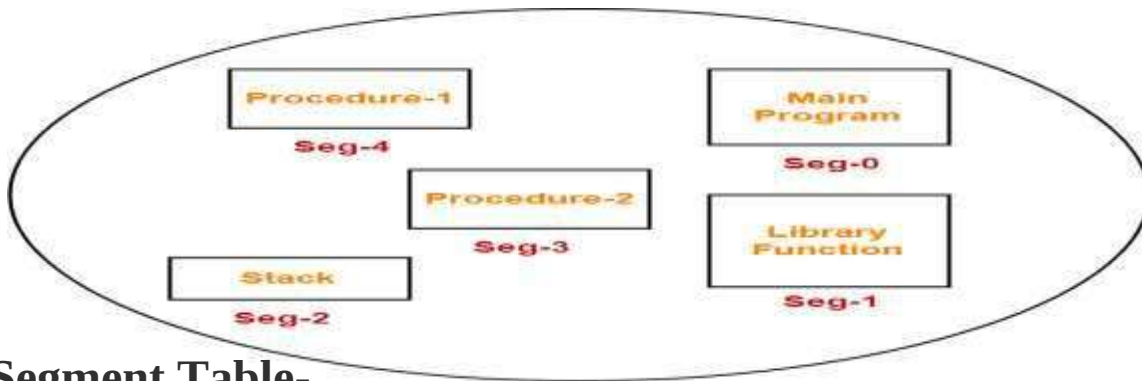
Characteristics-

Segmentation is a variable size partitioning scheme.

- In segmentation, secondary memory and main memory are divided into partitions of unequal size.
- The size of partitions depend on the length of modules.
- The partitions of secondary memory are called as **segments**.

Example-

Consider a program is divided into 5 segments as-



Segment Table-

- Segment table is a table that stores the information about each segment of the process.
- It has two columns.
- First column stores the size or length of the segment.
- Second column stores the base address or starting address of the segment in the main memory.
- Segment table is stored as a separate segment in the main memory.
- Segment table base register (STBR) stores the base address of the segment table.

For the above illustration, consider the segment table is-

	Limit	Base
Seg-0	1500	1500
Seg-1	500	6300
Seg-2	400	4300
Seg-3	1100	3200
Seg-4	1200	4700

Segment Table

Here,

- Limit indicates the length or size of the segment.
- Base indicates the base address or starting address of the segment in the main memory.

In accordance to the above segment table, the segments are stored in the main memory as-



Translating Logical Address into Physical Address-

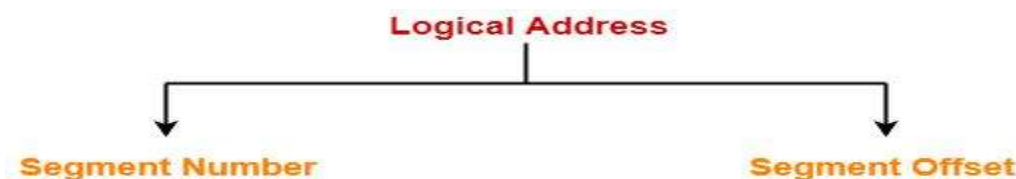
- CPU always generates a logical address.
- A physical address is needed to access the main memory.

Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of two parts-

1. Segment Number
2. Segment Offset



- Segment Number specifies the specific segment of the process from which CPU wants to read the data.
- Segment Offset specifies the specific word in the segment that CPU wants to read.

Step-02:

- For the generated segment number, corresponding entry is located in the segment table.
- Then, segment offset is compared with the limit (size) of the segment.

Now, two cases are possible-

Case-01: Segment Offset $\geq$ Limit

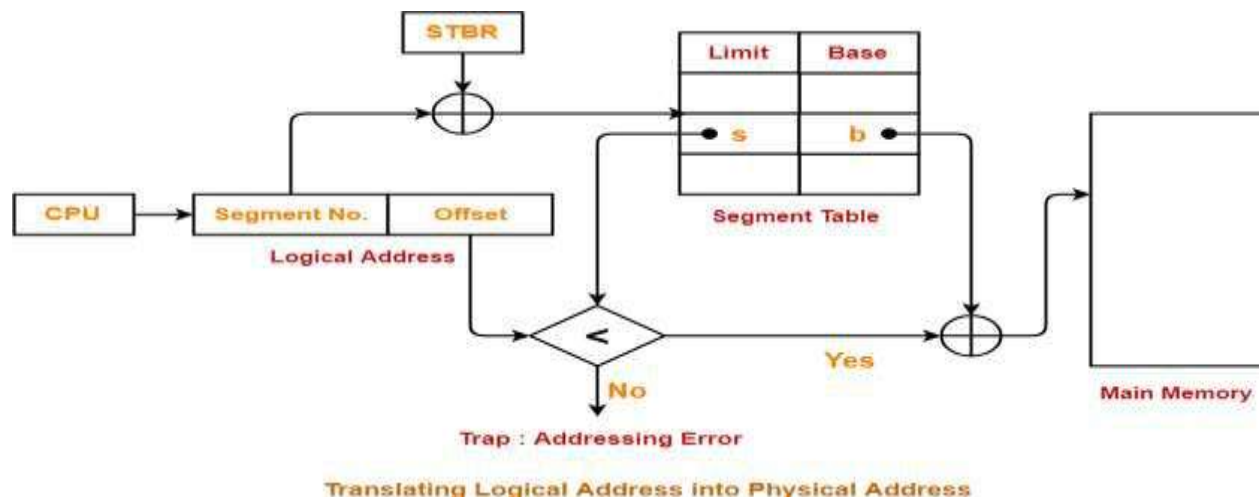
- If segment offset is found to be greater than or equal to the limit, a trap is generated.

Case-02: Segment Offset $<$ Limit

- If segment offset is found to be smaller than the limit, then request is treated as a valid request.
- The segment offset must always lie in the range $[0, \text{limit}-1]$,
- Then, segment offset is added with the base address of the segment.
- The result obtained after addition is the address of the memory location storing the required word.

Diagram-

The following diagram illustrates the above steps of translating logical address into physical address-



Advantages-

The advantages of segmentation are-

- It allows to divide the program into modules which provides better visualization.
- Segment table consumes less space as compared to **Page Table** in paging.
- It solves the problem of internal fragmentation.

Disadvantages-

The disadvantages of segmentation are-

- There is an overhead of maintaining a segment table for each process.
- The time taken to fetch the instruction increases since now two memory accesses are required.
- Segments of unequal size are not suited for swapping.
- It suffers from external fragmentation as the free space gets broken down into smaller pieces with the processes being loaded and removed from the main memory.

Differences between Paging and Segmentation

Now, we will cover the differences between Paging and Segmentation in the table given below:

Paging	Segmentation
Paging is a memory management technique where memory is partitioned into fixed-sized blocks that are commonly known as pages .	Segmentation is also a memory management technique where memory is partitioned into variable-sized blocks that are commonly known as segments .
With the help of Paging, the logical address is divided into a page number and page offset .	With the help of Segmentation, the logical address is divided into section number and section offset .
This technique may lead to Internal Fragmentation .	Segmentation may lead to External Fragmentation .
In Paging, the page size is decided by the hardware.	While in Segmentation, the size of the segment is decided by the user.
In order to maintain the page data, the page table is created in the Paging	In order to maintain the segment data, the segment table is created in the Paging
The page table mainly contains the base address of each page.	The segment table mainly contains the segment number and the offset.
This technique is faster than segmentation.	On the other hand, segmentation is slower than paging.
In Paging, a list of free frames is maintained by the Operating system.	In Segmentation, a list of holes is maintained by the Operating system.
In this technique, in order to calculate the absolute address page number and the offset both are required.	In this technique, in order to calculate the absolute address segment number and the offset both are required.

Differences between Contiguous and Non-Contiguous Memory Allocation.

Contiguous Memory Allocation	Non-Contiguous Memory Allocation
The contiguous Memory Allocation technique allocates one single contiguous block of memory to the process and memory is allocated to the process in a continuous fashion.	The non-Contiguous Memory allocation technique divides the process into several blocks and then places them in the different address space of the memory that is memory is allocated to the process in a non-contiguous fashion.
In this Allocation scheme, there is no overhead in the address translation while the execution of the process.	While in this scheme, there is overhead in the address translation while the execution of the process.
In Contiguous Memory Allocation, the process executes faster because the whole process is in a sequential block.	In Non-contiguous Memory allocation execution of the process is slow as the process is in different locations of the memory.
Contiguous Memory Allocation is easier for the Operating System to control.	The non-Contiguous Memory Allocation scheme is difficult for the Operating System to control.
In this, the memory space is divided into fixed-sized partitions and each partition is allocated only to a single process.	In this scheme, the process is divided into several blocks and then these blocks are placed in different parts of the memory according to the availability of memory space.
Contiguous memory allocation includes single partition allocation and multi-partition allocation .	Non-Contiguous memory allocation includes Paging and Segmentation .
In this type of memory allocation , generally, a table	In this type of memory allocation generally, a

is maintained by the operating system that maintains the list of all available and occupied partitions in the memory space.	table has to be maintained for each process that mainly carries the base addresses of each block that has been acquired by a process in the memory.
There is wastage of memory in Contiguous Memory allocation.	There is no wastage of memory in Non-Contiguous Memory allocation.
In this type of allocation, swapped-in processes are arranged in the originally allocated space.	In this type of allocation, swapped-in processes can be arranged in any place in the memory.

Difference Between Paging and Swapping

Let us cover the difference between Paging and Swapping in Operating System.

Paging	Swapping
Paging is a memory management technique in which computer stores and retrieves data from the secondary storage for use in the main memory.	Swapping is a technique that is used to temporarily remove the inactive programs from the main memory of the computer system
This technique allows more processes to reside in the main memory	Swapping allows fewer processes to reside in the main memory.
Paging follows non-contiguous memory management.	Swapping can be done without any memory management technique.
Paging is more flexible because in this there is the movement of pages of a process.	Swapping is less flexible as the entire process moves back and forth in the main memory and back store.
Paging occurs when some part of the process is transferred to the disk.	While Swapping occurs when the whole process is transferred to the disk.
For medium workloads paging technique is suitable.	For heavy workloads swapping technique is suitable.
This technique allows the memory address space of a process to be noncontiguous.	With the help of Swapping multiple processes can run in parallel in the Operating System
This technique helps to implement virtual memory.	Swapping helps the CPU to access processes faster.

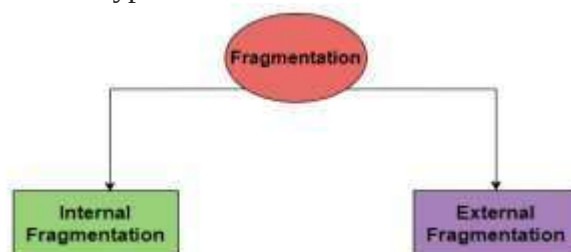
Fragmentation

In the storage system of the computer, as processes are loaded space is allocated to them and this space is divided on the basis of **fixed partitioning** (where the size of partitions is fixed before any process gets loaded into the memory) and **dynamic partitioning**(where the size of partitions is decided at the run time and it is mainly according to the size of the process).

Likewise, processes are loaded and then removed from the memory due to which the free space in the memory gets broken down into smaller pieces. This mainly creates small non utilized fragmented memory spaces that are so small due to which normal processes can not fit into them.

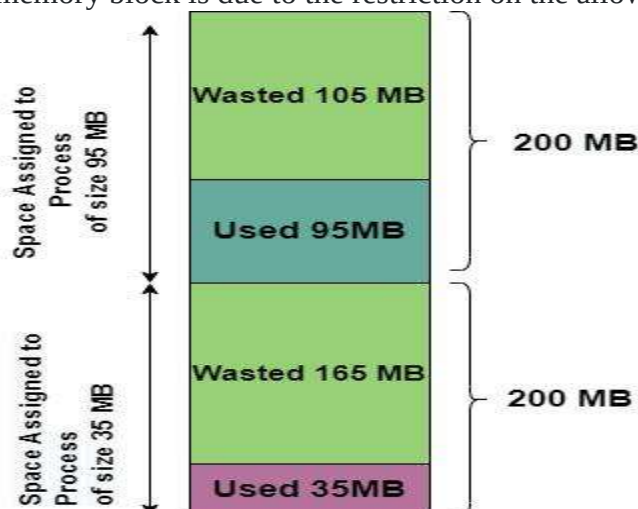
The problem due to which memory space is not getting utilized at all is commonly known as **Fragmentation** in the operating system.

Fragmentation is further divided into two types:



Internal Fragmentation

Internal Fragmentation is a problem that occurs when the process is allocated to a memory block whose size is more than the size of that process and due to which some part of the memory is left unused. Thus the space wasted inside the allocated memory block is due to the restriction on the allowed sizes of allocated blocks.

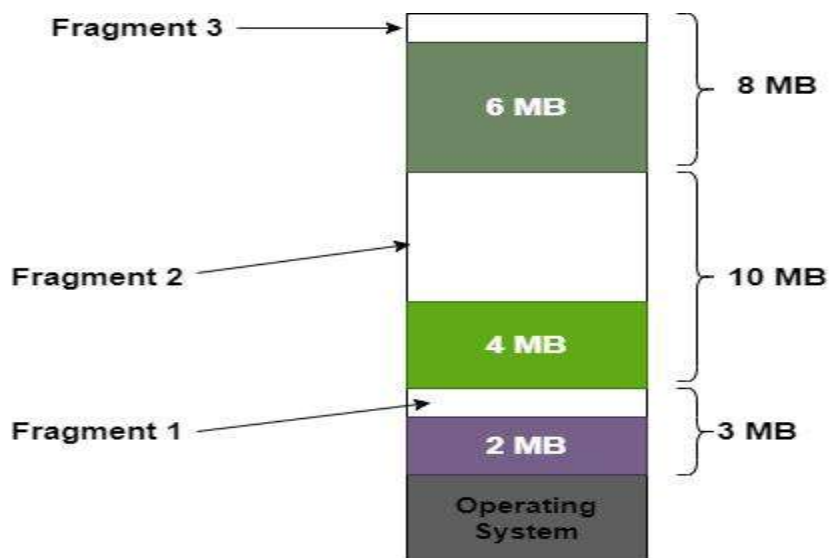


The above figure shows the Internal Fragmentation in Operating System

In the above diagram, the difference between memory allocated and required space or memory by the process is called **internal fragmentation**.

External Fragmentation

When the memory space in the system can easily satisfy the requirement of the processes, but this available memory space is **non-contiguous**, So it can't be utilized further. Then this problem is referred to as **External Fragmentation**.



The above Figure shows the External Fragmentation in Operating System

Suppose, we want to allocate memory to a process of size 8 MB and as In the above diagram, we can see that, there is enough space that is 9 MB to run a process of size 8MB but the memory that is fragments are not contiguous.

There is a solution for the problem of External Fragmentation and it is **Compaction**. The main goal is to shuffle the memory contents so as to place all the free memory together in one large block.

Also, Compaction is not always possible. Suppose if relocation is static and is done at the load time then in that case compaction cannot be done. Because compaction is only possible if the relocation is dynamic and is done at the execution time.

Difference Between Internal and External Fragmentation

Internal Fragmentation	External Fragmentation
Internal Fragmentation occurs when the memory blocks of fixed-size are allocated to the processes.	External Fragmentation occurs when the memory blocks of variable-size are allocated to the processes dynamically.
This type of fragmentation mainly occurs when the fixed size partition is assigned to a process whose size is less than the size of the partition due to which the rest of the space in the partition becomes unusable.	When the memory space in the system can easily satisfy the requirement of the processes, but this available memory space is non-contiguous , So it can't be utilized further.
The difference between allocated memory and memory required by a process is called Internal fragmentation.	Unused memory spaces between the non-contiguous memory fragments that are too small to serve a new process are called External fragmentation.
It mainly refers to the unused space in the partition that resides in the allocated region; as suggested by the name.	It mainly refers to the unused blocks of the memory that are not contiguous and hence are unable to satisfy the requirements of the process.
Best-fit block can be used to overcome the problem of Internal fragmentation.	Compaction, segmentation, and paging can be used to overcome the problem of External fragmentation.
Paging suffers from Internal fragmentation.	First-fit and best-fit suffer from external fragmentation.

Segmented Paging-

Segmented paging is a scheme that implements the combination of segmentation and paging.

Working-

In segmented paging,

- Process is first divided into segments and then each segment is divided into pages.
- These pages are then stored in the frames of main memory.
- A page table exists for each segment that keeps track of the frames storing the pages of that segment.
- Each page table occupies one frame in the main memory.
- Number of entries in the page table of a segment = Number of pages that segment is divided.
- A segment table exists that keeps track of the frames storing the page tables of segments.
- Number of entries in the segment table of a process = Number of segments that process is divided.
- The base address of the segment table is stored in the segment table base register.

Translating Logical Address into Physical Address-

CPU always generates a logical address.

- A physical address is needed to access the main memory.

Following steps are followed to translate logical address into physical address-

Step-01:

CPU generates a logical address consisting of three parts-

1. Segment Number
 2. Page Number
 3. Page Offset
- Segment Number specifies the specific segment from which CPU wants to read the data.
 - Page Number specifies the specific page of that segment from which CPU wants to read the data.
 - Page Offset specifies the specific word on that page that CPU wants to read.

Step-02:

- For the generated segment number, corresponding entry is located in the segment table.
- Segment table provides the frame number of the frame storing the page table of the referred segment.
- The frame containing the page table is located.

Step-03:

- For the generated page number, corresponding entry is located in the page table.
- Page table provides the frame number of the frame storing the required page of the referred segment.
- The frame containing the required page is located.

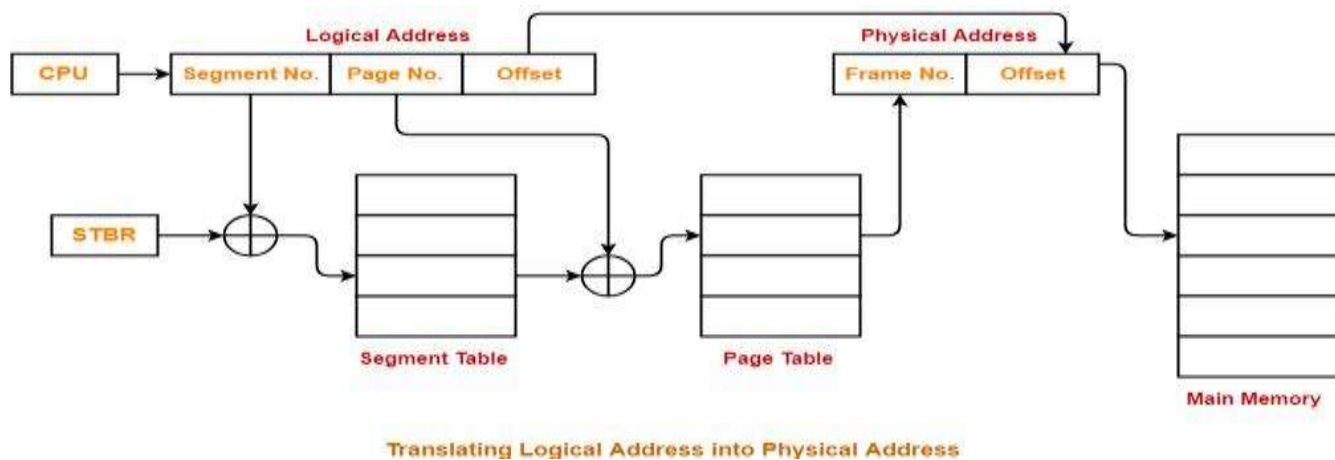
Step-04:

- The frame number combined with the page offset forms the required physical address.
- For the generated page offset, corresponding word is located in the page and read.



Diagram-

The following diagram illustrates the above steps of translating logical address into physical address-



Advantages-

The advantages of segmented paging are-

- Segment table contains only one entry corresponding to each segment.
- It reduces memory usage.
- The size of Page Table is limited by the segment size.
- It solves the problem of external fragmentation.

Disadvantages-

The disadvantages of segmented paging are-

- Segmented paging suffers from internal fragmentation.
- The complexity level is much higher as compared to paging.

Virtual Memory

Virtual Memory is a space where large programs can store themselves in form of pages while their execution and only the required pages or portions of processes are loaded into the main memory. This technique is useful as a large virtual memory is provided for user programs when a very small physical memory is there. Thus Virtual memory is a technique that allows the execution of processes that are not in the physical memory completely.

Virtual Memory mainly gives the illusion of more physical memory than there really is with the help of Demand Paging.

In real scenarios, most processes never need all their pages at once, for the following reasons :

- Error handling code is not needed unless that specific error occurs, some of which are quite rare.
- Arrays are often over-sized for worst-case scenarios, and only a small fraction of the arrays are actually used in practice.
- Certain features of certain programs are rarely used.

In an Operating system, the memory is usually stored in the form of units that are known as **pages**. Basically, these are atomic units used to store large programs.

Virtual memory can be implemented with the help of:-

1. Demand Paging
2. Demand Segmentation

The Need for Virtual Memory

Following are the reasons due to which there is a need for Virtual Memory:

- In case, if a computer running the Windows operating system needs more memory or RAM than the memory installed in the system then it uses a small portion of the hard drive for this purpose.
- Suppose there is a situation when your computer does not have space in the physical memory, then it writes things that it needs to remember into the hard disk in a swap file and that as virtual memory.

Benefits of having Virtual Memory

1. Large programs can be written, as the virtual space available is huge compared to physical memory.
2. Less I/O required leads to faster and easy swapping of processes.
3. More physical memory available, as programs are stored on virtual memory, so they occupy very little space on actual physical memory.
4. Therefore, the Logical address space can be much larger than that of physical address space.
5. Virtual memory allows address spaces to be shared by several processes.
6. During the process of creation, virtual memory allows: **copy-on-write** and **Memory-mapped files**

Advantages of Virtual Memory

Given below are the advantages of using Virtual Memory:

- Virtual Memory allows you to run more applications at a time.
- With the help of virtual memory, you can easily fit many large programs into smaller programs.
- With the help of Virtual memory, a multiprogramming environment can be easily implemented.
- As more processes should be maintained in the main memory which leads to the effective utilization of the CPU.
- Data should be read from the disk at the time when required.
- Common data can be shared easily between memory.
- With the help of virtual memory, speed is gained when only a particular segment of the program is required for the execution of the program.
- The process may even become larger than all of the physical memory.

Disadvantages of Virtual Memory

Given below are the drawbacks of using Virtual Memory:

- Virtual memory reduces the stability of the system.
- The performance of Virtual memory is not as good as that of RAM.
- If a system is using virtual memory then applications may run slower.
- Virtual memory negatively affects the overall performance of a system.

- Virtual memory occupies the storage space, which might be otherwise used for long-term data storage.
- This memory takes more time in order to switch between applications.

Demand Paging

Every process in the virtual memory contains lots of pages and in some cases, it might not be efficient to swap all the pages for the process at once. Because it might be possible that the program may need only a certain page for the application to run. Let us take an example here, suppose there is a 500 MB application and it may need as little as 100MB pages to be swapped, so in this case, there is no need to swap all pages at once.

The demand paging system is somehow similar to the paging system with swapping where processes mainly reside in the main memory(usually in the hard disk). Thus demand paging is the process that solves the above problem only by swapping the pages on Demand. This is also known as **lazy swapper**(It never swaps the page into the memory unless it is needed).

Swapper that deals with the individual pages of a process are referred to as **Pager**.

Demand Paging is a technique in which a page is usually brought into the main memory only when it is needed or demanded by the CPU. Initially, only those pages are loaded that are required by the process immediately. Those pages that are never accessed are thus never loaded into the physical memory.

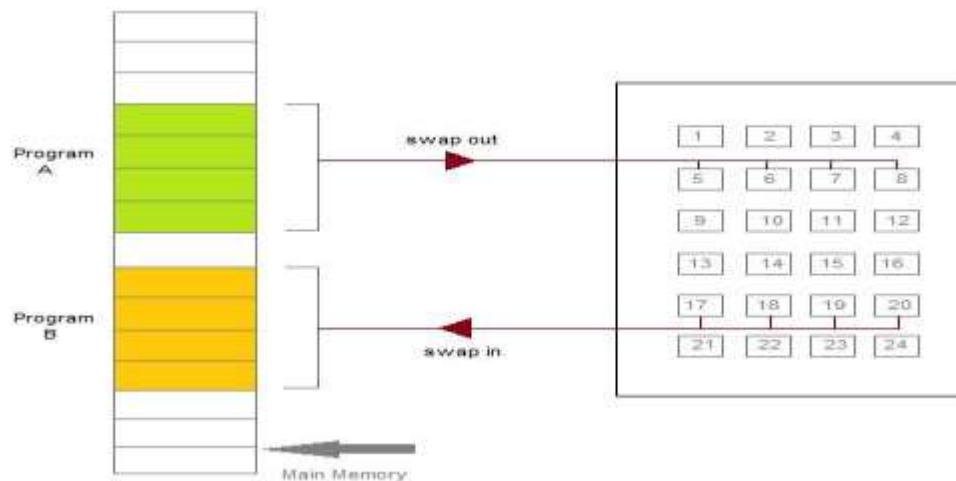


Figure: Transfer of a Paged Memory to the contiguous disk space. Whenever a page is needed? make a reference to it;

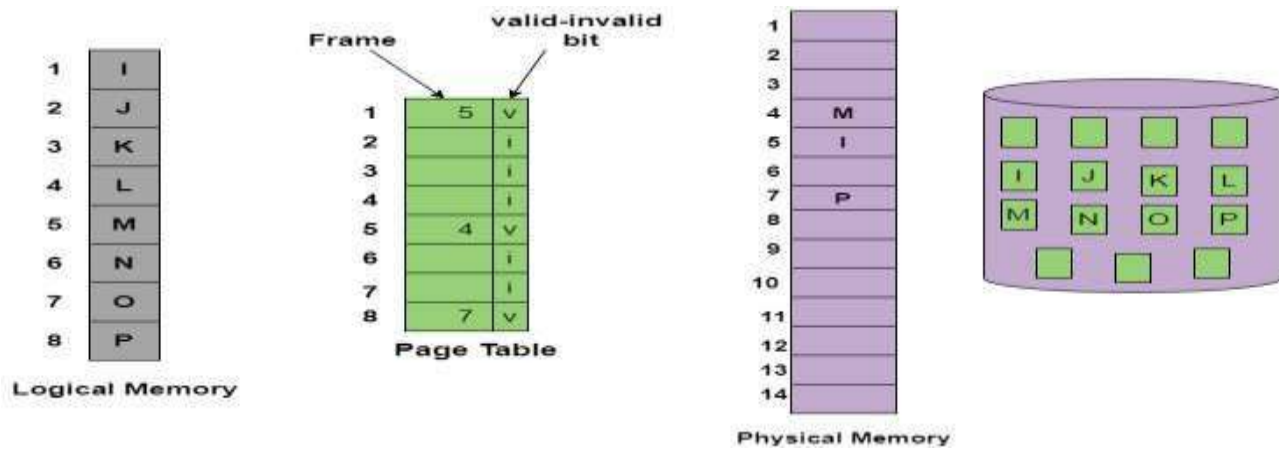
- If the reference is **invalid** then abort it.
- If the page is Not-in-memory then bring it to memory.

Valid-Invalid Bit

Some form of hardware support is used to distinguish between the pages that are in the memory and the pages that are on the disk. Thus for this purpose Valid-Invalid scheme is used:

- With each page table entry, a valid-invalid bit is associated(where **1** indicates **in the memory** and **0** indicates **not in the memory**)
- Initially, the valid-invalid bit is set to 0 for all table entries.
- 1. If the bit is set to "**valid**", then the associated page is **both legal and is in memory**.
- 2. If the bit is set to "**invalid**" then it indicates that the **page is either not valid** or the page is valid **but is currently not on the disk**.
- For the **pages** that are **brought into the memory**, the **page table is set as usual**.
- But for the **pages** that are **not currently in the memory**, the **page table** is either **simply marked as invalid** or it contains the **address of the page on the disk**.

During the translation of address, if the valid-invalid bit in the page table entry is 0 then it leads to **page fault**.



The above figure is to indicate the page table when some pages are not in the main memory.

How Demand Paging Works?

First of all the components that are involved in the Demand paging process are as follows:

- Main Memory
 - CPU
 - Secondary Memory
 - Interrupt
 - Physical Address space
 - Logical Address space
 - Operating System
 - Page Table
1. If a page is not available in the main memory in its active state; then a request may be made to the CPU for that page. Thus for this purpose, it has to generate an interrupt.
 2. After that, the Operating system moves the process to the blocked state as an interrupt has occurred.
 3. Then after this, the Operating system searches the given page in the Logical address space.
 4. And Finally with the help of the page replacement algorithms, replacements are made in the physical address space. Page tables are updated simultaneously.
 5. After that, the CPU is informed about that update and then asked to go ahead with the execution and the process gets back into its ready state.

When the process requires any of the pages that are not loaded into the memory, a page fault trap is triggered and the following steps are followed,

1. The memory address which is requested by the process is first checked, to verify the request made by the process.
2. If it is found to be invalid, the process is terminated.
3. In case the request by the process is valid, a free frame is located, possibly from a free-frame list, where the required page will be moved.
4. A new operation is scheduled to move the necessary page from the disk to the specified memory location. (This will usually block the process on an I/O wait, allowing some other process to use the CPU in the meantime.)
5. When the I/O operation is complete, the process's page table is updated with the new frame number, and the invalid bit is changed to valid.
6. The instruction that caused the page fault must now be restarted from the beginning.

Advantages of Demand Paging

The benefits of using the Demand Paging technique are as follows:

- With the help of Demand Paging, memory is utilized efficiently.
- Demand paging avoids External Fragmentation.
- Less Input/Output is needed for Demand Paging.
- This process is not constrained by the size of physical memory.
- With Demand Paging it becomes easier to share the pages.

- With this technique, portions of the process that are never called are never loaded.
- No compaction is required in demand Paging.

Disadvantages of Demand paging

Drawbacks of Demand Paging are as follows:

- There is an increase in overheads due to interrupts and page tables.
- Memory access time in demand paging is longer.

Pure Demand Paging

In some cases when initially no pages are loaded into the memory, pages in such cases are only loaded when are demanded by the process by generating page faults. It is then referred to as **Pure Demand Paging**.

- In the case of pure demand paging, there is not even a single page that is loaded into the memory initially. Thus pure demand paging causes the page fault.
- When the execution of the process starts with no pages in the memory, then the operating system sets the instruction pointer to the first instruction of the process and that is on a non-memory resident page and then in this case the process immediately faults for the page.
- After that when this page is brought into the memory then the process continues its execution, page fault is necessary until every page that it needs is in the memory.
- And at this point, it can execute with no more faults.
- This scheme is referred to as Pure Demand Paging: means never bring a page into the memory until it is required.

Page Fault

Page fault dominates more like an error. It mainly occurs when any program tries to access the data or the code that is in the address space of the program, but that data is not currently located in the RAM of the system.

- So basically when the page referenced by the CPU is not found in the main memory then the situation is termed as Page Fault.
- Whenever any page fault occurs, then the required page has to be fetched from the secondary memory into the main memory.

In case if the required page is not loaded into the memory, then a page fault trap arises

The page fault mainly generates an exception, which is used to notify the operating system that it must have to retrieve the "pages" from the virtual memory in order to continue the execution. Once all the data is moved into the physical memory the program continues its execution normally. The Page fault process takes place in the background and thus goes unnoticed by the user.

- The hardware of the computer tracks to the kernel and the program counter (PC) is generally saved on the stack. CPU registers store the information of the current state of instruction.
- An assembly program is started that usually saves the general registers and also saves the other volatile information to prevent the OS from destroying it.

Handling the Page Fault

Given below is the simple procedure to handle the page fault

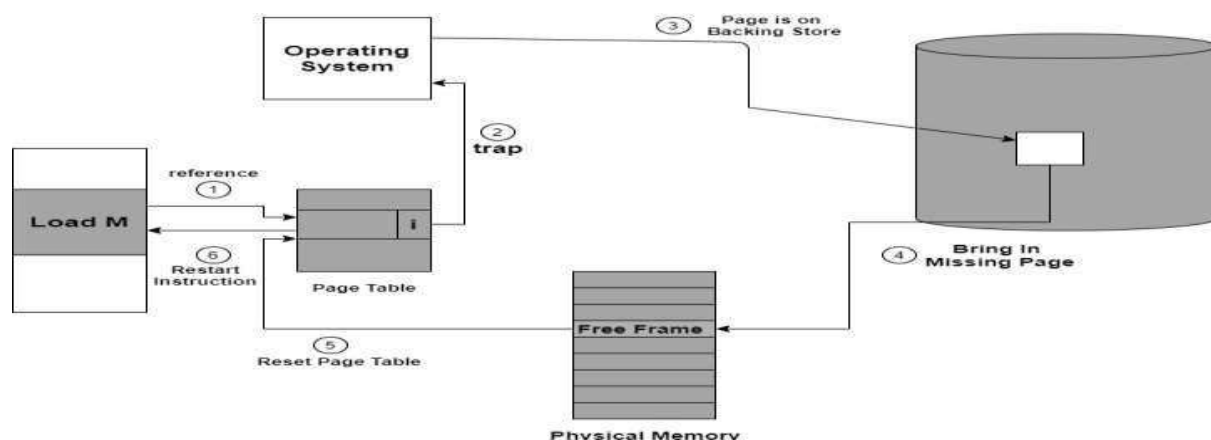


Figure: Steps to Handle the Page fault

If you will access a page that is marked as invalid then it also causes a Page Fault. Then the Paging hardware during translating the address through the page table will notice that the invalid bit is set that will cause a trap to the Operating system.

This trap is mainly the result of the failure of the Operating system in order to bring the desired page into memory.

Let us understand the procedure to handle the page fault as shown with the help of the above diagram:

1. First of all, internal table(that is usually the process control block) for this process in order to determine whether the reference was valid or invalid memory access.
2. If the reference is invalid, then we will terminate the process. If the reference is valid, but we have not bought in that page so now we just page it in.
3. Then we **locate the free frame list** in order to find the free frame.
4. Now a disk operation is scheduled in order to read the **desired page into the newly allocated frame**.
5. When the disk is completely read, then the **internal table is modified** that is kept with the process, and the page table that mainly indicates the page is now in memory.
6. Now we will restart the instruction that was interrupted due to the trap. Now the process can access the page as though it had always been in memory.

Page Replacement Algorithms:

As in Demand Paging, only certain pages of a process are loaded initially into the memory. This allows us to get more processes into memory at the same time. but what happens when a process requests for more pages and no free memory is available to bring them in. Following steps can be taken to deal with this problem :

1. Put the process in the wait queue, until any other process finishes its execution thereby freeing frames.
2. Or, remove some other process completely from the memory to free frames.
3. Or, find some pages that are not being used right now, move them to the disk to get free frames. This technique is called **Page replacement** and is most commonly used.

In this case, if a process requests a new page and supposes there are no free frames, then the Operating system needs to decide which page to replace. The operating system must use any page replacement algorithm in order to select the victim frame. The Operating system must then write the victim frame to the disk then read the desired page into the frame and then update the page tables. And all these require double the disk access time.

- Page replacement prevents the over-allocation of the memory by modifying the page-fault service routine.
- To reduce the overhead of page replacement a **modify bit (dirty bit)** is used in order to indicate whether each page is modified.
- This technique provides complete separation between logical memory and physical memory.

Page Replacement in OS

In Virtual Memory Management, Page Replacement Algorithms play an important role. The main objective of all the **Page replacement policies** is to decrease the maximum number of **page faults**.

Page Fault – It is basically a memory error, and it occurs when the current programs attempt to access the memory page for mapping into virtual address space, but it is unable to load into the physical memory then this is referred to as Page fault.

Basic Page Replacement Algorithm in OS

Page Replacement technique uses the following approach. If there is no free frame, then we will find the one that is not currently being used and then free it. A-frame can be freed by writing its content to swap space and then change the page table in order to indicate that the page is no longer in the memory.

1. First of all, find the location of the desired page on the disk.
2. Find a free Frame: a) If there is a free frame, then use it. b) If there is no free frame then make use of the page-replacement algorithm in order to select the victim frame. c) Then after that write the victim frame to the disk and then make the changes in the page table and frame table accordingly.

3. After that read the desired page into the newly freed frame and then change the page and frame tables.
4. Restart the process.

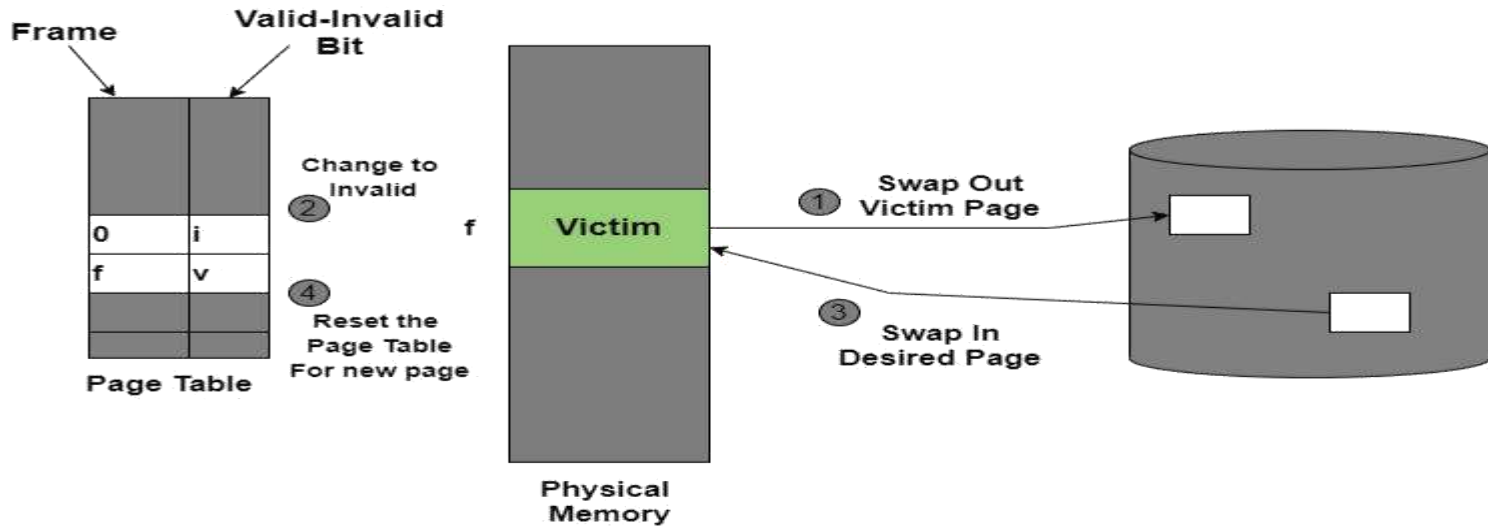
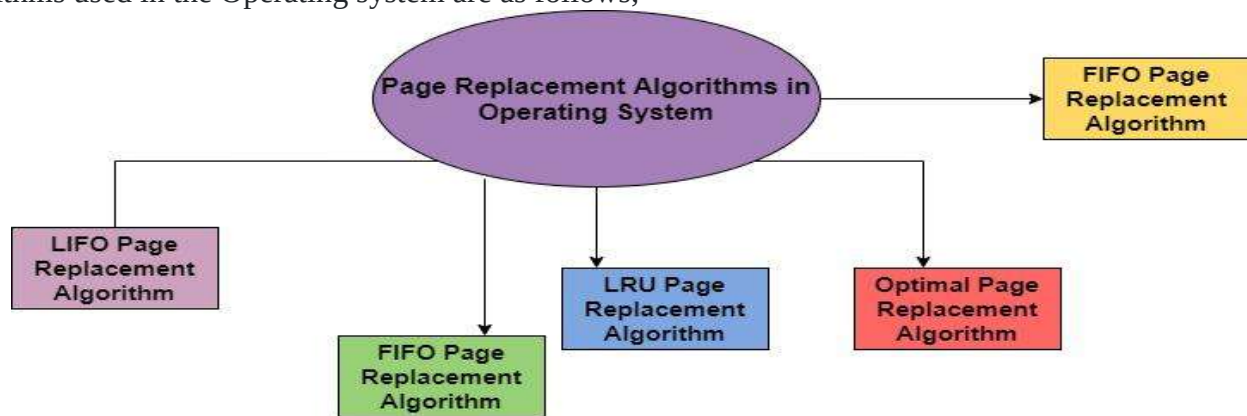


Figure: Page Replacement

Page Replacement Algorithms

This algorithm helps to decide which pages must be swapped out from the main memory in order to create a room for the incoming page. This Algorithm wants the lowest page-fault rate. Various Page Replacement algorithms used in the Operating system are as follows;



1. FIFO Page Replacement Algorithm

It is a very simple way of Page replacement and is referred to as First in First Out. This algorithm mainly replaces the oldest page that has been present in the main memory for the longest time.

- This algorithm is implemented by keeping the track of all the pages in the queue.
- As new pages are requested and are swapped in, they are added to the tail of a queue and the page which is at the head becomes the victim.
- This is not an effective way of page replacement but it can be used for small systems.

Advantages

- This algorithm is simple and easy to use.
- FIFO does not cause more overhead.

Disadvantages

- This algorithm does not make the use of the frequency of **last used time rather** it just replaces the Oldest Page.
- There is an increase in **page faults** as page frames increases.
- The performance of this algorithm is the worst.

2. LIFO Page Replacement Algorithm

This Page Replacement algorithm stands for "Last In First Out". This algorithm works in a similar way to the LIFO principle.

- In this, the newest page is replaced which is arrived at last in the primary memory
- This algorithm makes use of the stack for monitoring all the pages.

3. LRU Page Replacement Algorithm in OS

This algorithm stands for "Least recent used" and this algorithm helps the Operating system to search those pages that are used over a short duration of time frame.

- The page that has not been used for the longest time in the main memory will be selected for replacement.
- This algorithm is easy to implement.
- This algorithm makes use of the counter along with the even-page.

Advantages of LRU

- It is an efficient technique.
- With this algorithm, it becomes easy to identify the faulty pages that are not needed for a long time.
- It helps in Full analysis.

Disadvantages of LRU

- It is expensive and has more complexity.
- There is a need for an additional data structure.

4. Optimal Page Replacement Algorithm

This algorithm mainly replaces the page that will not be used for the longest time in the future. The practical implementation of this algorithm is not possible.

- Practical implementation is not possible because we cannot predict in advance those pages that will not be used for the longest time in the future.
 - This algorithm leads to less number of page faults and thus is the best-known algorithm
- Also, this algorithm can be used to measure the performance of other algorithms.

Advantages of OPR

- This algorithm is easy to use.
- This algorithm provides excellent efficiency and is less complex.
- For the best result, the implementation of data structures is very easy

Disadvantages of OPR

- In this algorithm future awareness of the program is needed.
- Practical Implementation is not possible because the operating system is unable to track the future request

5. Random Page Replacement Algorithm

As indicated from the name this algorithm replaces the page randomly. This Algorithm can work like any other page replacement algorithm that is LIFO, FIFO, Optimal, and LRU.

Page Replacement Algorithms Examples:

In an operating system that uses paging for memory management, a page replacement algorithm is needed to decide which page needs to be replaced when a new page comes in.

Page Fault – A page fault happens when a running program accesses a memory page that is mapped into the virtual address space but not loaded in physical memory.

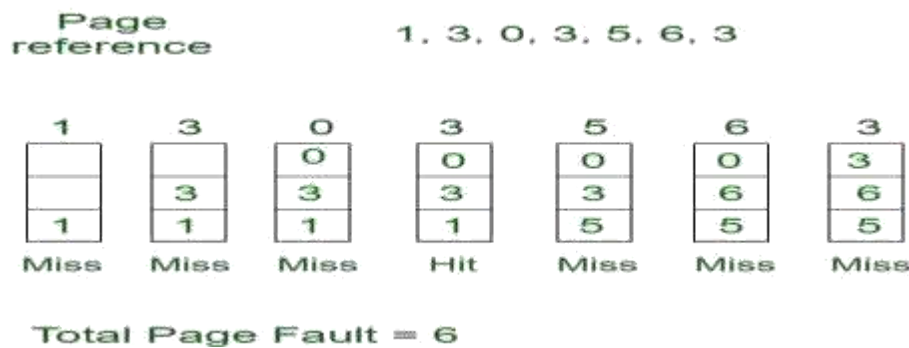
Since actual physical memory is much smaller than virtual memory, page faults happen. In case of a page fault, Operating System might have to replace one of the existing pages with the newly needed page. Different page replacement algorithms suggest different ways to decide which page to replace. The target for all algorithms is to reduce the number of page faults.

Page Replacement Algorithms :

1. First In First Out (FIFO) –

This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Example-1 Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames. Find the number of page faults.



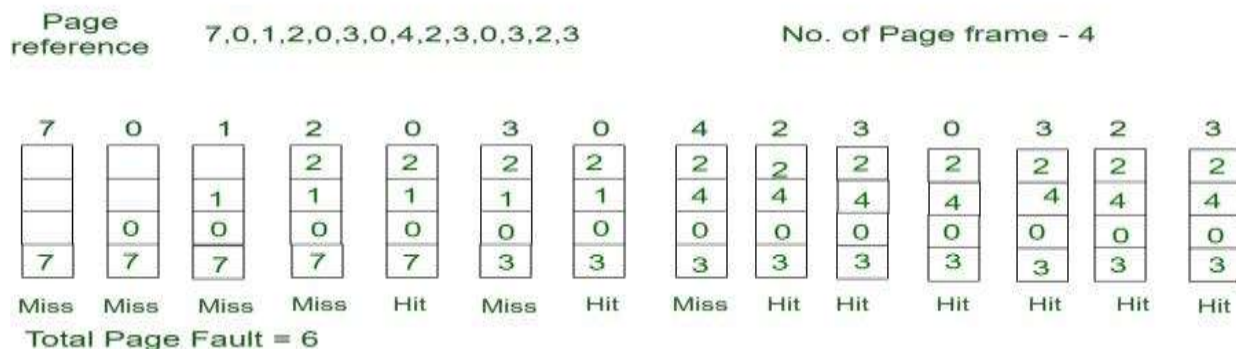
Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots —> **3 Page Faults**. when 3 comes, it is already in memory so —> **0 Page Faults**. Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. —> **1 Page Fault**. 6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3 —> **1 Page Fault**. Finally, when 3 come it is not available so it replaces 0 **1 page fault**

Belady's anomaly – Belady's anomaly proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm. For example, if we consider reference string 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4 and 3 slots, we get 9 total page faults, but if we increase slots to 4, we get 10-page faults.

2. Optimal Page replacement –

In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Example-2: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, with 4 page frame. Find number of page fault.



Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots —> **4 Page faults** 0 is already there so —> **0 Page fault**.

when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future.—> **1 Page fault**.

0 is already there so —> **0 Page fault..**

4 will takes place of 1 —> **1 Page Fault**.

Now for the further page reference string —> **0 Page fault** because they are already available in the memory. Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

3. Least Recently Used –

In this algorithm, page will be replaced which is least recently used. **Example-3** Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2 with 4 page frames. Find number of page faults.

Page
reference

7,0,1,2,0,3,0,4,2,3,0,3,2,3

No. of Page frame - 4

7	0	1	2	0	3	0	4	2	3	0	3	2	3
			2	2	2	2	2	2	2	2	2	2	2
		1	1	1	1	1	4	4	4	4	4	4	4
	0	0	0	0	0	0	0	0	0	0	0	0	0
7	7	7	7	7	3	3	3	3	3	3	3	3	3
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots —> **4 Page faults** 0 is already their so —> **0 Page fault.**

when 3 came it will take the place of 7 because it is least recently used —> **1 Page fault** 0 is already in memory so —> **0 Page fault.**

4 will takes place of 1 —> **1 Page Fault**

Now for the further page reference string —> **0 Page fault** because they are already available in the memory.

Question 2. Page Faults

Consider the following page reference string:

1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6.

How many page faults occur for the following page replacement algorithms assuming one, three, four, five, six, or seven frames?

- LRU replacement
- FIFO replacement
- Optimal replacement
- Do any of your examples exhibit Belady's anomaly? Explain.
- Which page replacement policy would you recommend? Why?

LRU:

3 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1:	1		4	5	1	7	2	-
Frame 2:		2	-	6	3	-	-	-
Frame 3:		3		1	2	-	6	1

15 faults

4 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1:	1			-	-	6	-
Frame 2:	2		-	-	-	-	-
Frame 3:		3		5	3	-	-
Frame 4:		4		6		7	1

10 faults

5 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1:	1		-		-		-
Frame 2:	2		-	-	-	-	-
Frame 3:	3			6		-	-
Frame 4:		4		3		-	-
Frame 5:				5		7	

8 faults

6 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 - - -

Frame 2: 2 - - - - -

Frame 3: 3 - - -

Frame 4: 4

Frame 5: 5 7

Frame 6: 6 - -

7 faults, which is the fewest we can have with 7 pages

FIFO:

3 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 4 6 3 -2-6

Frame 2: 2 - 1 2 - 7 1

Frame 3: 3 5 1 6 3

16 faults

4 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 - 5 3 - 1

Frame 2: 2 - 6 7 3

Frame 3: 3 2 - 6 -

Frame 4: 4 1 2 -

14 faults

5 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 - 6 - -

Frame 2: 2 - - 1 - -

Frame 3: 3 2 - -

Frame 4: 4 3 - -

Frame 5: 5 7

10 faults

6 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 - - 7

Frame 2: 2 - - - - 1

Frame 3: 3 - - 2

Frame 4: 4 3

Frame 5: 5

Frame 6: 6 - -

10 faults

Optimal:

3 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 - - 3 - -

Frame 2: 2 - - - 7 2 -

Frame 3: 3 4 5 6 - 1 6

11 faults

4 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 - - 7 1
 Frame 2: 2 - - - - -
 Frame 3: 3 - - -
 Frame 4: 4 5 6 - -
 8 faults

5 frames:

1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6

Frame 1: 1 - - - -
 Frame 2: 2 - - - - -
 Frame 3: 3 - - -
 Frame 4: 4 7
 Frame 5: 5 6 - -

7 faults, which is the best we can do

Thrashing

In case, if the page fault and swapping happens very frequently at a higher rate, then the operating system has to spend more time swapping these pages. This state in the operating system is termed thrashing. Because of thrashing the CPU utilization is going to be reduced.

Let's understand by an example, if any process does not have the number of frames that it needs to support pages in active use then it will quickly page fault. And at this point, the process must replace some pages. As all the pages of the process are actively in use, it must replace a page that will be needed again right away. Consequently, the process will quickly fault again, and again, and again, replacing pages that it must bring back in immediately. This high paging activity by a process is called thrashing.

During thrashing, the CPU spends less time on some actual productive work spend more time swapping.

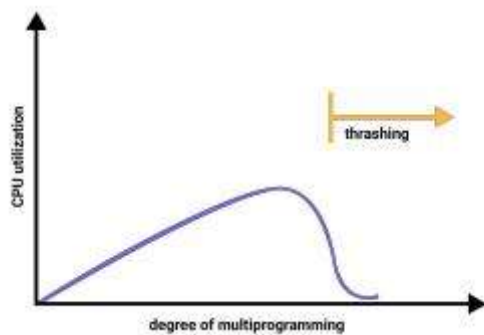


Figure: Thrashing

Causes of Thrashing

Thrashing affects the performance of execution in the Operating system. Also, thrashing results in severe performance problems in the Operating system.

When the utilization of CPU is low, then the process scheduling mechanism tries to load many processes into the memory at the same time due to which degree of Multiprogramming can be increased. Now in this

situation, there are more processes in the memory as compared to the available number of frames in the memory. Allocation of the limited amount of frames to each process.

Whenever any process with high priority arrives in the memory and if the frame is not freely available at that time then the other process that has occupied the frame is residing in the frame will move to secondary storage and after that this free frame will be allocated to higher priority process.

We can also say that as soon as the memory fills up, the process starts spending a lot of time for the required pages to be swapped in. Again the utilization of the CPU becomes low because most of the processes are waiting for pages.

Thus a high degree of multiprogramming and lack of frames are two main causes of thrashing in the Operating system.

Effect of Thrashing

At the time, when thrashing starts then the operating system tries to apply either the **Global page replacement** Algorithm or the **Local page replacement** algorithm. **Global Page Replacement**

The Global Page replacement has access to bring any page, whenever thrashing found it tries to bring more pages. Actually, due to this, no process can get enough frames and as a result, the thrashing will increase more and more. Thus the global page replacement algorithm is not suitable whenever thrashing happens.

Local Page Replacement

Unlike the Global Page replacement, the local page replacement will select pages which only belongs to that process. Due to this, there is a chance of a reduction in the thrashing. As it is also proved that there are many disadvantages of Local Page replacement. Thus local page replacement is simply an alternative to Global Page replacement.

Techniques used to handle the thrashing

As we have already told you the Local Page replacement is better than the Global Page replacement but local page replacement has many disadvantages too, so it is not suggestible. Thus given below are some other techniques that are used:

Working-Set Model

This model is based on the assumption of the locality. It makes the use of the parameter Δ in order to define the working-set window. The main idea is to examine the most recent Δ page reference. What locality is saying, the recently used page can be used again, and also the pages that are nearby this page will also be used?

1. Working Set

The set of the pages in the most recent Δ page reference is known as the working set. If a page is in active use, then it will be in the working set. In case if the page is no longer being used then it will drop from the working set Δ times after its last reference.

The working set mainly gives the approximation of the locality of the program.

The accuracy of the working set mainly depends on Δ what is chosen?

This working set model avoids thrashing while keeping the degree of multiprogramming as high as possible.

2. Page Fault Frequency

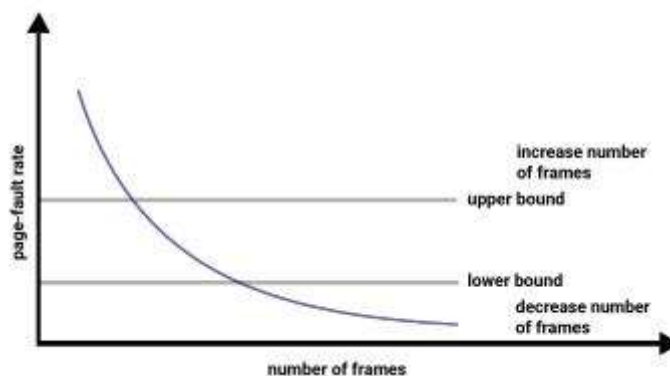


Figure: Page fault frequency

The working-set model is successful and its knowledge can be useful in preparing but it is a very clumpy approach in order to avoid thrashing. There is another technique that is used to avoid thrashing and it is Page Fault Frequency(PFF) and it is a more direct approach.

The main problem is how to prevent thrashing. As thrashing has a high page fault rate and also we want to control the page fault rate.

When the Page fault is too high, then we know that the process needs more frames. Conversely, if the page fault-rate is too low then the process may have too many frames.

We can establish upper and lower bounds on the desired page faults. If the actual page-fault rate exceeds the upper limit then we will allocate the process to another frame. And if the page fault rate falls below the lower limit then we can remove the frame from the process.

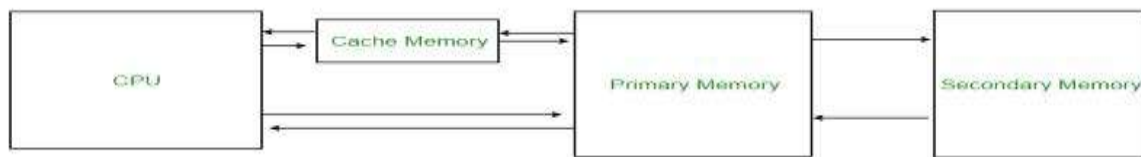
Thus with this, we can directly measure and control the page fault rate in order to prevent thrashing.

Cache Memory Organization

Cache Memory is a special very high-speed memory. The cache is a smaller and faster memory that stores copies of the data from frequently used main memory locations. There are various different independent caches in a CPU, which store instructions and data. The most important use of cache memory is that it is used to reduce the average time to access data from the main memory.

Characteristics of Cache Memory

- Cache memory is an extremely fast memory type that acts as a buffer between RAM and the CPU.
- Cache Memory holds frequently requested data and instructions so that they are immediately available to the CPU when needed.
- Cache memory is costlier than main memory or disk memory but more economical than CPU registers.
- Cache Memory is used to speed up and synchronize with a high-speed CPU.



Cache Memory

Levels of Memory

- **Level 1 or Register:** It is a type of memory in which data is stored and accepted that are immediately stored in the CPU. The most commonly used register is Accumulator, Program counter, Address Register, etc.
- **Level 2 or Cache memory:** It is the fastest memory that has faster access time where data is temporarily stored for faster access.
- **Level 3 or Main Memory:** It is the memory on which the computer works currently. It is small in size and once power is off data no longer stays in this memory.
- **Level 4 or Secondary Memory:** It is external memory that is not as fast as the main memory but data stays permanently in this memory.

Cache Performance

When the processor needs to read or write a location in the main memory, it first checks for a corresponding entry in the cache.

- If the processor finds that the memory location is in the cache, a Cache Hit has occurred and data is read from the cache.
- If the processor does not find the memory location in the cache, a **cache miss** has occurred. For a cache miss, the cache allocates a new entry and copies in data from the main memory, then the request is fulfilled from the contents of the cache.

The performance of cache memory is frequently measured in terms of a quantity called **Hit ratio**. Hit Ratio(H) = hit / (hit + miss) = no. of hits/total accesses

Miss Ratio = miss / (hit + miss) = no. of miss/total accesses = 1 - hit ratio(H)

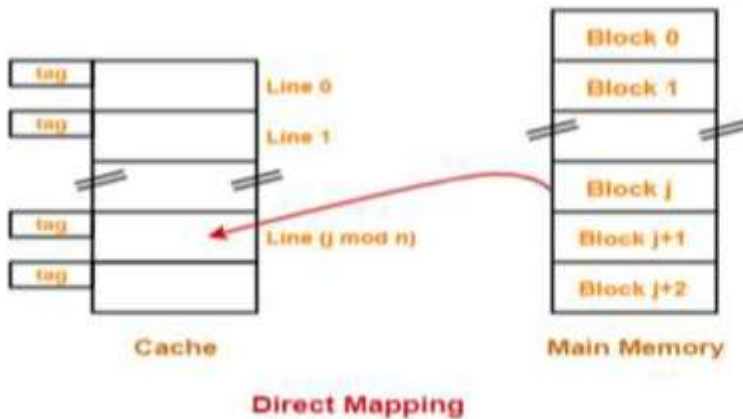
We can improve Cache performance using higher cache block size, and higher associativity, reduce miss rate, reduce miss penalty, and reduce the time to hit in the cache.

Cache Mapping

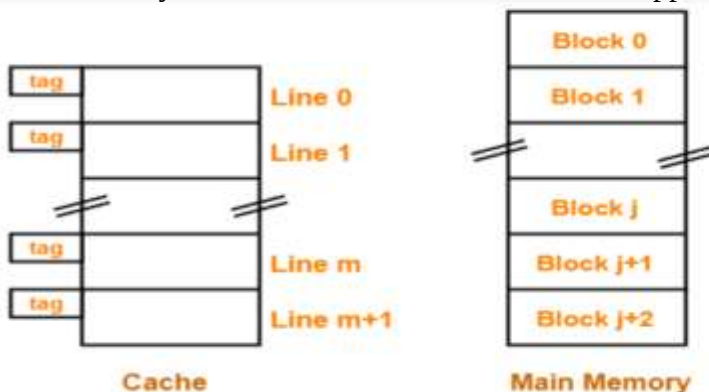
There are three different types of mapping used for the purpose of cache memory which is as follows:

- Direct Mapping
- Associative Mapping
- Set-Associative Mapping

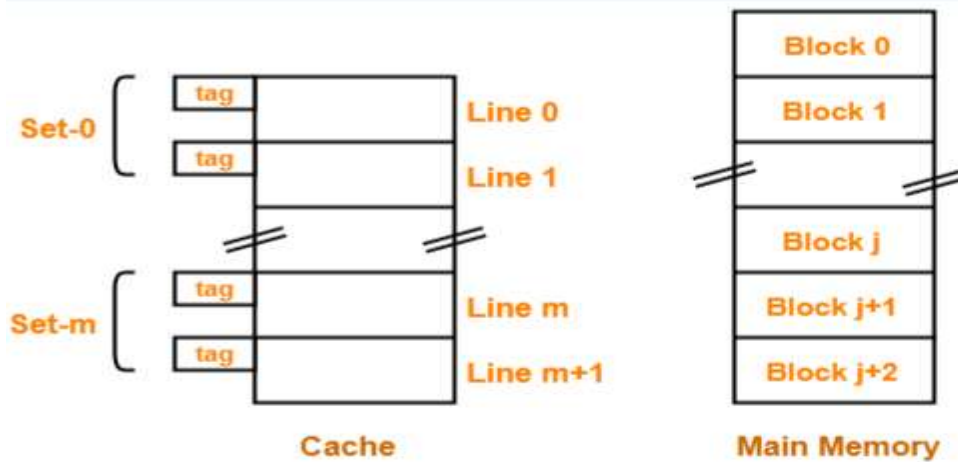
1. Direct Mapping: As the name suggests, it directly maps each block of main memory into the suitable cache line. It is also the simplest of all mapping techniques. In this, we link one main memory block to the cache line, and if another main memory block already acquires that cache line, it will search for the new one. The address part is split into two parts tag field or index field. Only the tag field is stored in a cache line, and the rest is stored in the main memory.



2. Associative Mapping: In this type of mapping, we use associative memory to store the address and content of the memory word. We can store any block in any line of Cache. Word id bits identify what bits are needed, and the tag becomes all the remaining bits. Because of this, any word can be placed anywhere in the cache memory. It is the most flexible and fastest mapping technique.



3. Set Associative Mapping: This is the upgraded form of direct mapping as it removes the drawbacks of direct mapping. It mainly targets the problem of thrashing, i.e., if the main memory block wants to point to some location of cache memory, but some other main memory block is already pointing to that. It solves this problem by following the method that instead of pointing one memory block line to one cache memory line, we make a set of some lines, and through it, we can point to the required cache line. It allows each word in the set can have more than one main memory location.



Locality of Reference: refers to a phenomenon in which a computer program tends to access same set of memory locations for a particular time period. In other words, **Locality of Reference** refers to the tendency of the computer program to access instructions whose addresses are near one another. The property of locality of reference is mainly shown by loops and subroutine calls in a program.

There are mainly two localities of reference.

1. **Temporal Locality of Reference:** This LRU(Least Recent Algorithm) will be used. Whenever a web page fault happens inside a phrase, it will no longer best load phrase in the primary reminiscence. However, the entire web page fault might be loaded because the spatial locality of reference rule says that if you are referring to any phrase, the subsequent phrase might be referred to in its check-in, that's why we load the entire web page desk so the entire block might be loaded.
2. **Spatial Locality of Reference:** In this, there is a chance that an element can be present in the close proximity of the reference point and will become closer and closer when searched again.

Application of Cache Memory

Here are some of the applications of Cache Memory.

1. **Primary Cache:** A primary cache is always located on the processor chip. This cache is small and its access time is comparable to that of processor registers.
2. **Secondary Cache:** Secondary cache is placed between the primary cache and the rest of the memory. It is referred to as the level 2 (L2) cache. Often, the Level 2 cache is also housed on the processor chip.
3. **Spatial Locality of Reference:** Spatial Locality of Reference says that there is a chance that the element will be present in close proximity to the reference point and next time if again searched then more close proximity to the point of reference.
4. **Temporal Locality of Reference:** Temporal Locality of Reference uses the Least recently used algorithm will be used. Whenever there is page fault occurs within a word will not only load the word in the main memory but the complete page fault will be loaded because the spatial locality of reference rule says that if you are referring to any word next word will be referred to in its register that's why we load complete page table so the complete block will be loaded.

Advantages of Cache Memory

- Cache Memory is faster in comparison to main memory and secondary memory.
- Programs stored by Cache Memory can be executed in less time.
- The data access time of Cache Memory is less than that of the main memory.
- Cache Memory stored data and instructions that are regularly used by the CPU, therefore it increases the performance of the CPU.

Disadvantages of Cache Memory

- Cache Memory is costlier than primary memory and secondary memory.
- Data is stored on a temporary basis in Cache Memory.
- Whenever the system is turned off, data and instructions stored in cache memory get destroyed.
- The high cost of cache memory increases the price of the Computer System.